QuantCode-Eval: Benchmarking Quantitative Strategy Code Reproduction from Finance Papers

Wenhao Lu[1†] **Ziqi Yuan**[1†] **Hao Wu**[1†] **Dunhong Jin**[2] **Chuan Wu**[1]

[†]Equal contribution

[1]Department of Computer Science [2]Faculty of Business and Economics
The University of Hong Kong, Hong Kong SAR

{whlu, zqyuanss}@connect.hku.hk {wuhao55, cwu, dhjin}@hku.hk

Abstract

Large language models (LLMs) have enabled impressive coding agents for various domains, driving growing interest in applying them to automate domain-specific workflows. In ***quantitative strategy development***, a domain with high stakes and high potential returns, reproducing executable strategy code from financial papers and reports is valuable for both academic research and industrial gains. However, it remains unclear whether LLMs can faithfully ***reproduce such domain-knowledge-intensive strategy code*** from finance papers. These papers often omit critical implementation details, while high-quality strategy code is scarce as pretraining data for latest LLMs. This paper presents QuantCode-Eval[1], a benchmark for evaluating the ability of LLMs to reproduce strategies from 30 leading quantitative finance papers published (or released) between 2010 and 2026 into executable trading strategies. To support rigorous evaluation, each reproduction task includes around 20 property-based executable checkers that return a binary pass/fail result. We conduct extensive evaluation on 7 proprietary and open-source LLMs with coding-agent scaffolds. The results show that even the best-performing setup, Claude Opus 4.7 with Claude Code, achieves a 43% pass@5 rate when reproducing code that satisfies all properties. We also perform a fine-grained analysis of violated properties by jointly examining their semantics and the agent trajectories. Our findings reveal diverse failures in strategy-sensitive aspects: (i) maintaining temporal consistency, (ii) correctly interpreting and formalizing strategy definitions, and (iii) preserving the intended signal direction during trade execution.

1 Introduction

In quantitative finance, academic literature serves as a rich and continuous source of market factors and systematic trading ideas. Financial practitioners routinely translate paper-described ideas into executable strategies to test across proprietary datasets and diverse trading frequencies [1]. This finance paper-to-strategy pipeline is so pervasive that published trading strategies experience an average 58% decay in return predictability post-publication, as practitioners actively reproduce and deploy them to arbitrage these factors away [2].

However, translating mathematically dense, narrative-driven finance papers into executable, backtestable code remains a manual and error-prone process, since quantitative research demands absolute semantic precision. Faithful strategy reproduction extends far beyond generating syntactically correct Python code. It requires strict semantic alignment across signal construction, temporal data processing, portfolio formation, and backtesting logic. In this domain, even a seemingly trivial code

[1]Benchmark, code, and leaderboard available at `www.quantcodeeval.cloud`

Preprint.

deviation—such as a single-period index shift causing look-ahead bias, or a misapplied portfolio weighting scheme—can completely invalidate a strategy's economic meaning and real-world viability.

Recent advances in LLM-based code generation have enabled AI systems to tackle increasingly complex programming tasks, motivating a growing set of benchmarks for general code generation [3–6]. More recent benchmarks have extended evaluation toward research reproduction [7–12], reflecting growing interest in whether LLM agents can translate scientific papers into executable artifacts. However, quantitative strategy reproduction remains under-explored within existing paper-to-code evaluations. Despite its practical importance and methodological complexity, there is still no systematic benchmark for evaluating whether LLM agents can reproduce strategies from finance papers as executable backtesting code. Building such a benchmark presents three distinct challenges:

First, *trading strategies impose strict semantic alignments, yet the most accessible signals of correctness are often only final return or Sharpe ratio*. A trading strategy consists of data processing, signal construction, portfolio formation, order execution, and backtesting (§ 2). To faithfully reproduce a finance paper, each stage must satisfy fine-grained semantic specifications. Coarse end-to-end metrics or trading records cannot determine whether an implementation matches the paper or localize errors.

Figure 1 illustrates this gap with an example: the double-shrinkage global minimum variance portfolio strategy paper [13]. Although the code reproduced by Opus 4.7 achieves a Sharpe ratio close to the paper's result, it deviates from the intended portfolio construction logic. The strategy performs shrinkage by regularizing the covariance matrix as $S_\lambda = \lambda S_n + (1-\lambda)I$, where S_n is the sample covariance matrix, I is the identity matrix, and λ controls the shrinkage strength. Crucially, the paper optimizes λ with a data-driven loss estimator, whereas the buggy code fixes $\lambda = 0.5$. When the optimized value happens to be close to 0.5, the incorrect code can still produce a similar Sharpe ratio.

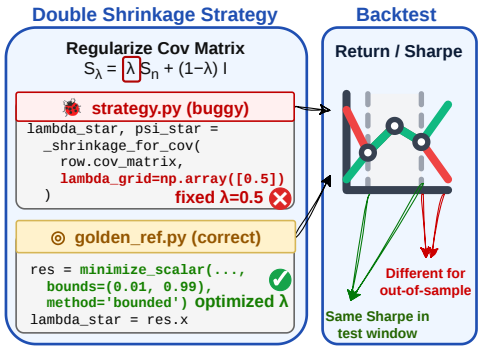


Figure 1: Matching Sharpe ratio on one dataset does not ensure a correct strategy.

Second, *implicit domain knowledge is involved in quantitative finance papers.* Finance research papers often omit implementation details that are standard in practice [14, 15], such as avoiding look-ahead bias, accounting for survivorship bias (e.g., ignoring delisted securities by forward-filling prices, or miscalculating delisting returns), market impact, and adjusting prices for dividends and other corporate actions [16, 17]. Yet these details are often error-prone in code generation and can determine whether a reproduced strategy is truly valid. Thus, the benchmark must check whether the domain-specific rules are followed.

Third, *high-quality user instructions are needed as joint input.* In practice, users rarely implement a finance paper in isolation. Many papers report multiple strategy variants in their experiments, such as quintile versus decile portfolio sorts [18], and users must specify which variant they intend to reproduce. Users may also impose extra constraints, such as using a proprietary data source. To better reflect this, the benchmark should take both the paper and an explicit user instruction as input for strategy reproduction, where the instruction resolves ambiguous implementation choices in the paper. Reproduction correctness should then be evaluated against both the paper and the clarified user intent.

We present QuantCode-Eval, a new benchmark to evaluate whether LLMs can faithfully reproduce correct quantitative strategy code. The benchmark contains 30 papers, either published in top-tier finance venues or identified by finance experts as providing useful insight for strategy generation. It spans six strategy categories validated by quantitative experts: Momentum, Mean Reversion, Factor Investing, Volatility Management, Event-Driven Trading, and Systematic Allocation. For each paper, we provide expert-curated instructions that clarify critical under-specified details and simulate customization requirements in practical deployments. To better evaluate the reproduced strategies, we collaborate with quantitative experts to extract properties, i.e., precise, checkable requirements that a correct strategy implementation must satisfy. The properties cover both general constraints for valid backtesting and paper-specific strategy logic. For example, a general property requires portfolio construction at day t to use only information available before day t, preventing look-ahead bias. A paper-specific property requires the shrinkage parameter in Figure 1 to be optimized using

the formula specified by the paper, rather than replaced by a fixed constant. Then, each property is synthesized into an executable checker with diverse test inputs, enabling fine-grained correctness evaluation and failure analysis beyond end-to-end backtest metrics.

We evaluate latest LLMs and agentic coding frameworks on QuantCode-Eval. Even the strongest Claude Opus 4.7 with Claude Code agent scaffold achieves only a 43% pass@5 rate under our property-based evaluation. Furthermore, we identify several recurring failure patterns in strategy reproduction, including difficulty preserving strategy-critical temporal consistency, insufficient attention to paper-specific parameters and textual specifications, and a tendency to overwrite or simplify paper-specific definitions with more familiar prior knowledge. Our main contributions include:

• QuantCode-Eval: a benchmark built from 30 quantitative finance papers, primarily drawn from top-tier finance venues and supplemented with a few recent expert-selected working papers, to evaluate whether state-of-the-art (SOTA) LLMs can faithfully reproduce quantitative trading strategies across six major strategy categories.

• A property-based evaluation framework for quantitative strategy reproduction. Several expert-defined properties are to be checked by each strategy reproduction task.

• A comprehensive evaluation of SOTA LLMs and coding agents on QuantCode-Eval, showing that current systems remain unreliable at faithfully reproducing quantitative strategies and exhibit recurring failures in strategy-sensitive semantics, particularly temporal consistency, faithful formalization of paper-specific definitions, and preservation of signal direction during execution.

2 Preliminaries: Quantitative Strategy Workflow and Taxonomy

A quantitative strategy is a trading logic that relies on market data collection, mathematical models, and statistical analysis to systematically explore asset prices, market rules, and trading signals. The strategy can formulate investment decisions such as trading time, position management, and risk management [19, 20, 14]. In practice, a strategy is typically implemented as an executable and backtestable program. As shown in Figure 2, a strategy implementation generally consists of six stages [21–25]: 1) *data loading*, where market data from different assets and venues, such as closing prices and returns, are used as input; 2) *feature engineering*, where raw historical data are preprocessed and transformed into informative features; 3) *model fitting*, using the processed features for modeling and analysis, such as regression or machine learning; 4) *signal generation*, where trading signals are derived from analytical results and strategy hypotheses; 5) *portfolio construction*, translating signals into position sizing; and 6) *execution*, where trades are executed with risk management, producing outputs such as excess return and Sharpe ratio.

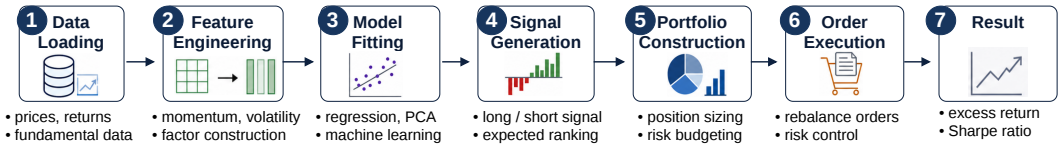


Figure 2: A six-stage quantitative strategy pipeline. A strategy implementation loads market data, constructs features, fits models, generates signals, constructs portfolios, and executes trades. The strategy is evaluated through backtesting and paper trading for further refinement.

Quantitative strategies differ in their trading rationales and in the sources of alpha they exploit, where alpha refers to predictable, realizable excess returns [1, 26, 27]. We categorize the strategies into six classes — Momentum [28], Mean Reversion [29], Factor [30], Volatility [31], Event-Driven [19], and Systematic Allocation [13]. Our taxonomy is drawn from practitioner and textbook classifications [32, 22], and validated by experienced quantitative trading experts. Our benchmark includes 5 selected papers in each strategy category (§3.1 *Paper selection*).

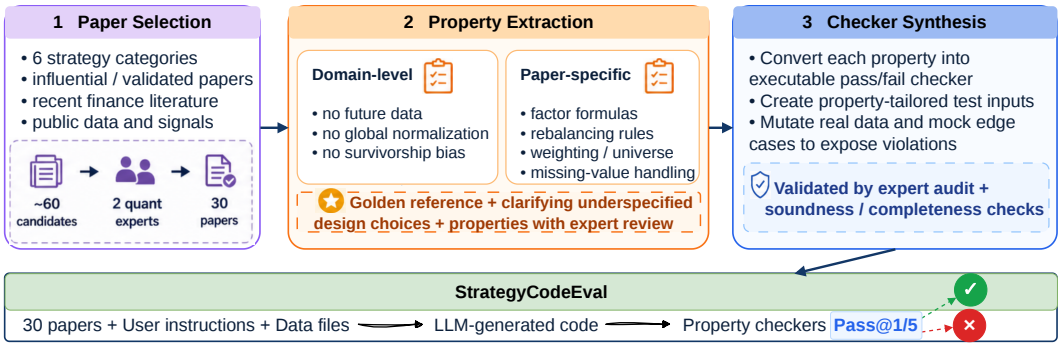


Figure 3: Benchmark construction workflow of QuantCode-Eval.

3 QuantCode-Eval

3.1 Benchmark Construction

In this section, we describe the human-LLM collaboration framework used to construct QuantCode-Eval benchmark. As shown in Figure 3, the construction workflow consists of three stages: paper selection, property extraction, and checker synthesis.

Task setup. Each benchmark task consists of a selected research paper, a user instruction, and the data files required to reproduce the target strategy. As finance papers often leave implementation choices under-specified, we offer a user instruction for each paper to clarify the reproduction task. The instruction serves two purposes: 1) it clarifies strategy-related details that are omitted or ambiguous in the original paper, and 2) it specifies the typed function signatures through which our checkers call and evaluate the submitted implementation. Crucially, the instruction clarifies what the strategy should do, but not how to implement it. It therefore avoids leaking low-level implementation details that an LLM could exploit as a shortcut. Appendix D.3 provides an example user instruction, and Appendix C documents the metadata and data-source provenance for each paper reproduction task.

Given these inputs, each evaluated LLM coding agent is required to generate a backtestable Python implementation of the strategy. We evaluate agents under both pass@1 and pass@5 settings [4]. pass@1 gives each agent one run per task, while pass@5 gives up to five independent runs and counts the task as solved if any run faithfully reproduces the strategy.

Paper selection. We construct a category-balanced and empirically reproducible paper collection using four criteria: (i) coverage of all six strategy categories; (ii) inclusion of work that is either academically influential or has been validated in practice; (iii) publication within the past sixteen years; and (iv) availability of the required signals and underlying data through the standard public sources used in the paper. The final collection of $5 \times 6 = 30$ papers is constructed as follows. We first manually shortlist ten papers per category (primarily from top finance venues), including the *Journal of Finance*, the *Journal of Financial Economics*, and other leading finance journals, prioritizing influential and category-relevant papers with feasible data requirements. This step yields approximately sixty candidate papers. Two quantitative trading experts, each with more than ten years of industry experience, then select the five most representative papers in each category that also satisfy criterion (iv). The final collection is summarized in Appendix C.

Potential training-data contamination [33] is admitted by most benchmarks based on public papers [8, 10]. For closed-source models, we cannot directly inspect their training data and therefore cannot fully rule out this risk. However, the observed performance does not suggest that models can simply recall existing implementations, as even the strongest LLMs still fail on many reproduction tasks. For open-weight models, we inspect the public paper-related resources and find that, although a few papers have public code, most available code is incomplete or non-Python, and thus not directly reusable for our tasks. Moreover, our reproduction tasks have new user instructions. Therefore, contamination is unlikely to be the main driver of our results. Moreover, future versions of QuantCode-Eval will continue to include newer papers and track public resources associated with each paper.

Table 1: Six representative properties from *Short-term Momentum* [18]. The *Stage* column refers to which of the 6 quantitative strategy stages introduced in §2 are covered by each property.

ID	Property Name	Description & Expected Behavior	Stage
Type A — Domain-level properties			
A3	no_future_data_access	Portfolio construction at month t must use only information available before month t. Using data from the same month or any future period would leak future information into the strategy.	All
A2	no_global_normalize	Return breakpoints must be recalculated separately each month using only the current month's stock universe. Using a single normalization rule or fixed breakpoints across all months incorrectly mixes information from different time periods.	S2
A5	no_survivorship_bias	Universe must be reconstructed each month from live CRSP or Compustat records, including stocks up to their delisting date. Using a fixed list of currently active stocks introduces survivorship bias.	S1, S2
Type B — Paper-specific properties			
B3	nasdaq_volume_adjustment	NASDAQ trading volume must be adjusted before turnover calculation because historical NASDAQ volumes were overstated. The paper applies different scaling factors for different periods: $\div 2.0$ before 2001-02, $\div 1.8$ from 2001-02 to 2001-12, $\div 1.6$ from 2002-01 to 2003-12, and no adjustment after 2004-01.	S2
B6	conditional_sort_order	Stocks are first sorted into return deciles (using NYSE breakpoints), then within each return decile into turnover deciles. Independently assigning return and turnover decile labels over the full universe, and then intersecting the two labels, form portfolios different from the required sequential sort.	S2, S4
B11	value_weighted_portfolios	Both the STMOM and STREV* portfolios must be value-weighted (market-cap proportional) on both long and short legs. Equal weighting is the primary failure mode and produces substantially different returns.	S5

Property extraction. For each selected paper, the two experts independently annotate properties that reflect the fine-grained requirements a valid reproduction must satisfy. Disagreements are identified through cross-checking and then resolved through joint discussion before the properties are finalized.

As shown in Table 1, we group the properties into two classes. *Domain-level properties* (henceforth *Type-A*) are correctness constraints that any valid and backtestable quantitative strategy must satisfy. They restrict the code behavior at each stage, from data loading to execution. For example, they include avoiding using future information during signal generation, preserving the causal order between signal generation and trade execution, and constructing a universe free of survivorship bias. A complete list of Type-A properties is provided in Appendix D.1. On the other hand, *paper-specific properties* (henceforth *Type-B*) capture the design choices that distinguish a paper's strategy from others in the same category, such as the factor formulas, rebalancing frequency, position-sizing rules, and universe selection criteria. Type-B properties of a sample paper task are listed in Appendix D.2.

In our extraction process, Type-A properties are derived from empirical studies [34, 22, 35–38] of common strategy implementation errors and expert experience. These properties are shared across the benchmark because they come from pipeline stages common to most strategies. Type-B properties, however, are paper-specific and must capture the critical formulas and design choices of each strategy. Since finance papers often describe strategies in a scattered and under-specified manner, we do not construct Type-B properties solely from the papers. Instead, for each paper, we first implement the strategy based on the paper and validate it with expert review, treating the resulting implementation as the golden reference. The golden reference serves as a clarified and executable interpretation of the paper strategy. By running and inspecting it, we calibrate Type-B properties against the actual strategy pipeline and identify implicit or under-specified design choices, such as rebalancing frequency, universe filtering, and missing-value handling. This improves the coverage of properties since they can be derived from the full executable pipeline rather than only from steps explicitly stated in the paper. It also improves property quality by grounding each requirement in validated implementation behavior. Finally, we include these clarified design choices in the user instruction, so that LLMs under test do not need to infer Type-B requirements under-specified by the paper.

Checker synthesis. We convert each annotated property into an executable checker that returns a binary pass/fail result. Each checker is equipped with test inputs tailored to the property being evaluated. For simple properties, such as fixed parameter values or basic computations, we generate inputs for the corresponding code snippet or individual pipeline stage. For more complex properties that involve interactions across multiple stages, we construct specialized raw data inputs (e.g., prices and fundamental data) based on expert experience. Specifically, beginning with real data,

experts identify data mutations that are likely to expose property violations, such as different market periods, stock universes of varying sizes, and mock data. For example, consider the checker for `A3 no_future_data_access` in Appendix D.4. To detect whether future-month information is improperly used in current-month trading decisions, the checker mocks an artificially large return value into a future month. It then examines the portfolio return of the preceding month. If the earlier-month return changes significantly after the mutation, it indicates that future information has leaked into the strategy pipeline, revealing a look-ahead bias violation. The checkers are constructed and validated by two CS PhD students, one CS postdoc, and multiple quantitative experts. The concrete evaluation of precision and recall is given in § 4.5.

3.2 Benchmark Statistics

The benchmark contains 30 papers, five from each of the six strategy categories introduced in §2. Of these, 63% appear in the *Journal of Finance*, *Journal of Financial Economics*, or *Review of Financial Studies*, while the remainder come from the *Journal of Financial and Quantitative Analysis*, *Management Science*, and others. The average citation count per paper has reached 711 by the beginning of May, 2026. The publication years range from 2010 to 2026.

Each paper has a median of 8 Type-A and 10 Type-B properties, and the corpus contains 572 property checkers in total. Golden-reference implementations span 177 to 1,190 lines of Python code with a median of 478 lines. More comprehensive per-paper statistics (e.g., category, venue, citation count, property counts, and golden-reference lengths) are listed in Appendix C and Appendix H.

4 Experimental Evaluation

This section evaluates SOTA LLMs and coding agents on QuantCode-Eval. We first report aggregate pass@1 and pass@5 success rate for reproducing code that satisfies all properties in § 4.2. We then analyze LLMs' performance across property types in § 4.3. Next, we use agent trajectories to study where and why reproduction failures arise in § 4.4. Finally, we validate the benchmark itself in § 4.5.

4.1 Evaluation Setup

We evaluate both closed-source and open-weight LLMs on QuantCode-Eval under the two complementary configurations summarized in Table 2. Because modern code-generation performance depends not only on the underlying LLM but also on the surrounding coding-agent scaffold, our evaluation covers both native agents and open-source agent scaffolds. In the *closed-source* setting, except for Qwen 3.6 Plus, each model is evaluated through its official, native scaffold, i.e., Claude Opus and Sonnet through Claude Code, and GPT through the OpenAI Codex CLI [39]. In the *open-weight* setting, we use OpenCode [40], a widely adopted open-source agent scaffold, and vary only the model backend, thereby enabling controlled comparisons.

Table 2: Coding-agent systems evaluated on QuantCode-Eval. All 7 LLMs are evaluated under OpenCode as the standardized scaffold; the four closed-source models are additionally evaluated in their native commercial coding-agent scaffolds.

Model	Developer	Scaffold
Closed-source		
Claude Opus 4.7	Anthropic	Claude Code / OpenCode
Claude Sonnet 4.6	Anthropic	Claude Code / OpenCode
GPT-5.5	OpenAI	Codex CLI / OpenCode
Qwen 3.6 Plus	Alibaba	OpenCode
Open-weight		
GLM 5.1	Zhipu AI	OpenCode
MiniMax M2.7	MiniMax	OpenCode
DeepSeek V4 Pro Preview	DeepSeek	OpenCode

For each benchmark sample, the coding agent receives a finance paper, user instruction, and strategy-specific data files. Each run is executed in an isolated Python virtual environment with a fixed toolchain, preventing code and result leakage across runs. During reproduction, the agent has no access to the golden reference or property checkers. For each sample, all submissions (i.e., LLM-reproduced strategy implementations) are evaluated with the same data inputs and runtime environment. Failed executions, timeouts, missing outputs, function-signature violations, and incorrect results are counted as failed submissions. For detailed analysis, we archive each run's submitted code, tool-call traces, intermediate execution logs, checker outputs, and backtest metrics. We submitted these artifacts with the container images and reproduction instructions to OpenReview. Hardware specifications, software versions, and per-agent decoding parameters are provided in Appendix E.

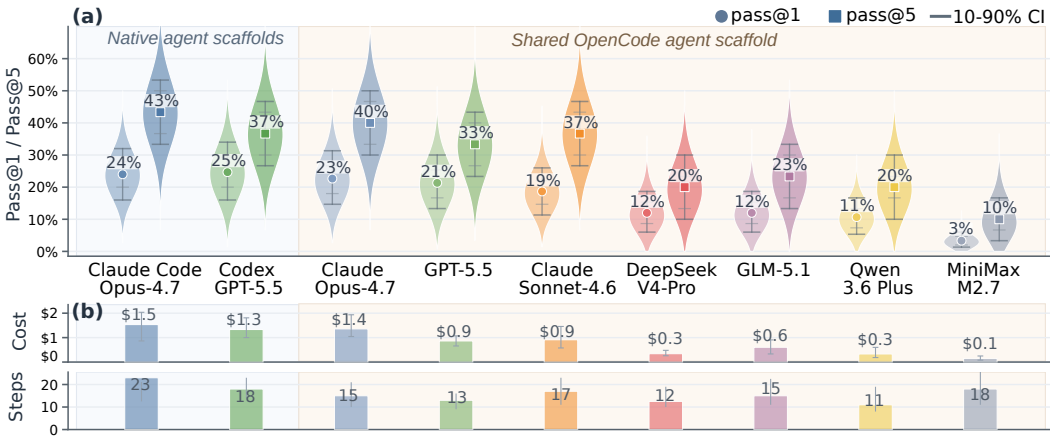


Figure 4: Main results and resource usage on QuantCode-Eval. (a) The top subfigure reports observed pass@1 and pass@5 for each coding agent under the full checker suite, following the pass@k convention used in prior benchmarks [4, 5]. Pass@1 is the fraction of successful attempts over 30 tasks with 5 attempts per task, and pass@5 is the fraction of tasks solved by at least one of the five attempts. Violin densities show task-bootstrap distributions obtained by resampling the 30 task-level values; vertical intervals mark the 10–90% bootstrap range, with shorter caps at the 25th and 75th percentiles. (b) The bottom subfigure reports per-attempt resource usage for attempts that produced a strategy implementation. Bars show medians and error bars show interquartile ranges for accounted USD cost and normalized interaction steps per reproduction attempt.

4.2 Main Results

Closed-source LLMs remain stronger, while open-weight models incur lower cost. As shown in Figure 4, closed-source models generally outperform open-weight models across the benchmark. However, under our API-token cost accounting, they typically incur a 2–3× higher cost than open-weight models. Among the closed-source models, Claude Opus 4.7 achieves the strongest overall performance, with 24% pass@1 and 43% pass@5, outperforming GPT-5.5 in our evaluation. Among open-weight models, DeepSeek V4 and GLM 5.1 are the strongest performers.

Native and open-source scaffolds show slightly different empirical performance. To assess the role of coding-agent scaffolds, we further evaluate all models under the OpenCode framework. Figure 4 shows that native coding agents slightly improve model pass@1 and pass@5 by up to 4%. Further analysis of interaction steps shows that, under OpenCode, the agent typically takes fewer interaction steps and appears to perform less extensive exploration, which can partially explain the reproduction quality gap. Nevertheless, even without their native scaffolds, closed-source models continue to maintain an evident advantage over open-weight models.

4.3 Evaluation Breakdown by Properties

We analyze property-level performance to understand where coding agents succeed or fail.

Property pass rates form a graded ladder across coding agents. Figure 5(a) plots the distribution of property pass rates over each coding agent's 150 attempts. Two features stand out. First, the distributions are centered around a single dominant peak. Few attempts cluster at either all-pass or near-zero pass performance, and most fall in between. Second, the distribution shifts steadily from the strongest closed-source LLMs to the open-weight LLMs, suggesting that the checker suite can differentiate capability levels across the evaluated models and agents without becoming either too easy or too hard. Appendix F extends this analysis to individual paper reproduction tasks. We observe that, for every individual task, the properties still separate stronger and weaker LLMs and agents.

Agents still fail on finance-critical properties at multiple levels. As Figure 5(b) shows, even on domain-level Type-A properties, which capture the basic requirements of a correct strategy implementation, the strongest coding agent reaches only an 86.2% pass rate, while the weakest

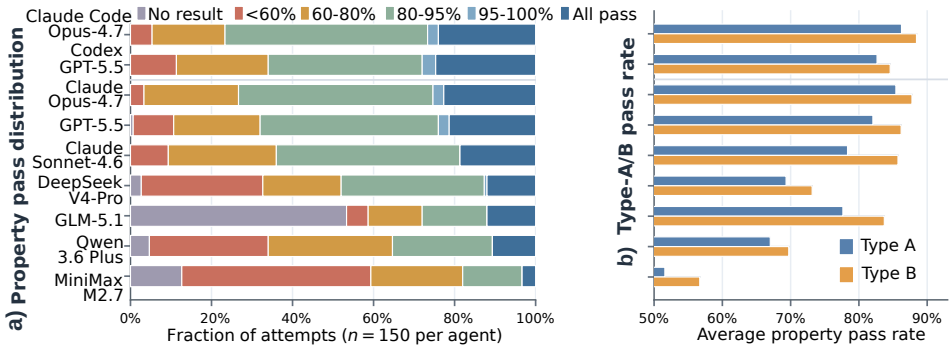


Figure 5: Breakdown analysis with property-level checkers. (a) Distribution of per-attempt property pass rates for each agent over 150 attempts per agent (30 tasks $\times$ 5 attempts). Attempts that time out or fail to produce executable checker results are counted as *No result* in the figure; these outcomes are analyzed separately in Appendix F. (b) Average checker pass rate for runnable reproductions, reported separately for Type-A domain-level checkers and Type-B paper-specific checkers.

Table 3: The grouped failure category of 36 Type-B properties with average failure rates above 50%. See Table 10 in Appendix F for all property details and classification rules.

Type-B Property Failure Group	# / % in 36	Failure Rate (All Agents)	Failure Rate (Opus4.7+GPT5.5)
Temporal consistency failures	17 / 47%	72.0%	64.7%
Definition implementation failures	16 / 44%	72.0%	68.1%
Trading direction failures	3 / 9%	79.5%	73.3%

reaches just 51.6%. For each paper-specific Type-B property, we measure its average failure rate as the fraction of attempts that fail the corresponding checker, aggregated across all agents and strategy reproduction attempts.

As Table 3 shows, among the 36 Type-B properties with average failure rates above 50%, three dominant finance-specific failure modes are: 1) *temporal consistency failures*, where the implementation violates required timing requirements, including lag structure, expanding windows, warmup periods, or out-of-sample timing, for example by using contemporaneous predictors instead of lagged ones; 2) *definition implementation failures*, which fail to preserve paper-specific formulas, parameters, or model components, for example by replacing a paper-specified formula with a library default; and 3) *trading-direction failures*, which reverse or distort how signals are mapped into trades, for example by implementing a high-minus-low portfolio as lowest-quintile minus highest-quintile instead of highest-quintile minus lowest-quintile.

4.4 Failure Analysis: Why LLMs Fail to Reproduce Strategies

We inspect the full trajectories of submissions that violate at least one property requirement, identifying each violation's root cause and earliest error stage in Figure 6. Details of full failure analysis, including how we classify failures, are provided in Appendix G. We identify four main failure types.

Retrieval failure (RC-A): the agent misses relevant evidence from the paper or instruction. Agents usually retrieve the main sections of long papers reasonably well, but retrieval failures still occur when a property depends on information spread across multiple parts of the paper. In finance papers, formulas are often not introduced as clean, self-contained specifications: assumptions, variable meanings, and timing conditions may be scattered across nearby paragraphs. As a result, the agent may recover the main equation but miss a critical textual constraint, or rely on an informal description without connecting it to the corresponding formal definition. These failures are relatively rare (4.6%), but even a single missed condition can materially change the reproduced trading rule.

Comprehension failure (RC-B): the agent retrieves the evidence but misinterprets it. The agent's reasoning trace includes the relevant content, but the agent misstates the corresponding financial or data-processing concept. For example, a paper [41] defines *cumulative residual* as the residual accumulated from the start of the measurement period through time t, but the agent treats it

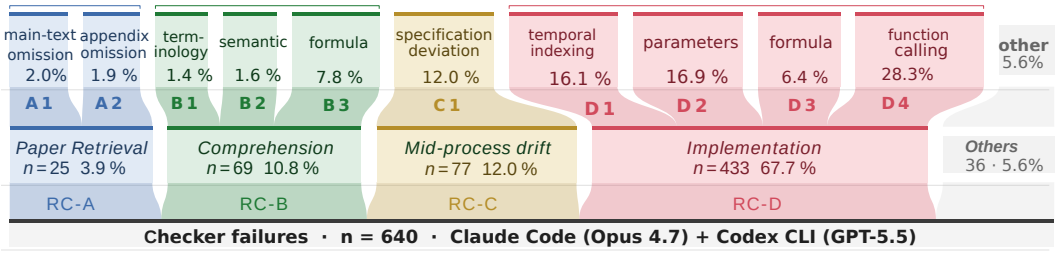


Figure 6: Fine-grained failure root cause (**RC**) analysis of Claude Code with Claude Opus 4.7 and Codex CLI with GPT-5.5. We inspect each failed trajectory to identify the earliest error stage and root cause of each property violation, and then group violations by failure type.

as a rolling sum over a fixed look-back window. The distinction matters because a rolling window drops older observations, whereas a cumulative residual keeps aggregating from the beginning of the period. In another case, the paper [20] normalizes portfolio weights using the trace of a shrunk covariance matrix, $\mathrm{tr}(\hat{\Sigma}) = \sum_i \hat{\Sigma}_{ii}$, i.e., the sum of the asset-level variance terms after shrinkage. However, the agent misinterprets this equation and uses the original covariance matrix.

Mid-process drift (RC-C): the agent initially understands the property but later deviates from it. This type is common for properties with many factors or multiple algorithmic layers. The agent understands the requirement correctly early in the trajectory, but later loses fidelity by simplifying, substituting, or partially dropping it during reasoning or planning. For instance, a paper [42] specifies a multi-factor strategy built with 20 boosted P-Trees, i.e., tree-based weak learners combined into an ensemble, yet the agent later decides to implement only 5 trees in its reasoning trajectory. Similarly, it may recognize that volatility should be estimated with an asymmetric GJR-GARCH model according to the a paper [43], but later replace it with simple rolling variance.

Implementation failure (RC-D): the final code does not correctly realize the requirement. The agent states the requirement correctly throughout the reasoning process, but the submitted code fails to realize it faithfully. Common bugs include temporal misalignment, where a trading strategy should only use information available at the decision time. For example, a paper [42] requires using the signal at time t to predict the next-period return r_{t+1}. The agent correctly aligns the target return with `ret.shift(-1)`, but then dates the output at $t+1$ rather than t, reintroducing a timing mismatch. We also observe more general programming errors. For instance, in one momentum sample [44], the agent correctly defines market equity, `me`, as absolute price times shares outstanding, but then constructs an intermediate table that reads the `me` column before initialization.

4.5 Evaluating Checker Precision and Recall

Before evaluating model-reproduced code, we develop and stress-test the checkers using only the golden references and their targeted mutants. The main goal is to improve the recall (i.e., reduce false negatives), where an incorrect implementation is incorrectly accepted. We generate executable mutants [45–47] from each golden reference using property-specific mutation templates. By construction, each mutant preserves the required function interface expected by the corresponding checker, but introduces a non-trivial semantic violation. Specifically, the mutator perturbs an expression on the data-flow path to the output, ensuring that the targeted property (e.g., long–short orientation, lookahead freeness, or a documented hyperparameter binding) is semantically, rather than merely syntactically, violated. During development, we use 32 such mutants per checker to identify missed violations and guide further refinement. After this stage, the checker suite is fixed. For empirical evaluation of the recall, we generate 64 brand new mutants for each refined checker. On average, the refined checkers detect 98.7% of new domain-level mutants and 87.1% of new paper-specific mutants, suggesting reasonable recall under our stress test.

To evaluate the precision, we further apply the fixed checkers to LLM-reproduced strategies and manually audit the checker-rejected submissions. We have not observed any false positives yet.

5 Conclusion

This paper presents QuantCode-Eval, a benchmark for evaluating whether LLMs can reproduce executable quantitative trading strategies from 30 leading finance papers. Each task includes a high-quality instruction, manually annotated strategy properties, and validated property-based checkers. Experiments show that current coding agents still struggle to faithfully implement domain-intensive financial strategies. Our trajectory-level analyses reveal recurring failures in temporal consistency, strategy formalization, and preservation of trading direction, highlighting a large gap between generating executable code and faithfully reproducing expert financial strategies.

References

[1] Ernest P Chan. *Quantitative trading: how to build your own algorithmic trading business*. John Wiley & Sons, 2026.

[2] R David McLean and Jeffrey Pontiff. Does academic research destroy stock return predictability? *The Journal of Finance*, 71(1):5–32, 2016.

[3] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[6] Zihan Zheng, Zerui Cheng, Zeyu Shen, Shang Zhou, Kaiyuan Liu, Hansen He, Dongruixuan Li, Stanley Wei, Hangyi Hao, Jianzhu Yao, et al. Livecodebench pro: How do olympiad medalists judge llms in competitive programming? *arXiv preprint arXiv:2506.11928*, 2025.

[7] Minju Seo, Jinheon Baek, Seongyun Lee, and Sung Ju Hwang. Paper2code: Automating code generation from scientific papers in machine learning. *arXiv preprint arXiv:2504.17192*, 2025.

[8] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.

[9] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.

[10] Tianyu Hua, Harper Hua, Violet Xiang, Benjamin Klieger, Sang T Truong, Weixin Liang, Fan-Yun Sun, and Nick Haber. Researchcodebench: Benchmarking llms on implementing novel machine learning research code. *arXiv preprint arXiv:2506.02314*, 2025.

[11] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, et al. Mlgym: A new framework and benchmark for advancing ai research agents. *arXiv preprint arXiv:2502.14499*, 2025.

[12] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.

[13] Taras Bodnar, Nestor Parolya, and Erik Thorsén. Two is better than one: Regularized shrinkage of large minimum variance portfolios. *Journal of machine learning research*, 25(173):1–32, 2024.

[14] Albert J. Menkveld, Anna Dreber, Felix Holzmeister, Jürgen Huber, Magnus Johannesson, Michael Kirchler, Michael Razen, Utz Weitzel, et al. Nonstandard errors. *The Journal of Finance*, 2024.

[15] Kewei Hou, Chen Xue, and Lu Zhang. Replicating anomalies. *The Review of Financial Studies*, 33(5):2019–2133, 2020.

[16] Robert D. Arnott, Campbell R. Harvey, and Harry Markowitz. A backtesting protocol in the era of machine learning. *The Journal of Financial Data Science*, 1(1):64–74, 2019.

[17] Marcos López de Prado. The 10 reasons most machine learning funds fail. *The Journal of Portfolio Management*, 44(6):120–133, 2018.

[18] Mamdouh Medhat and Maik Schmeling. Short-term momentum. *The Review of Financial Studies*, 35(3):1480–1526, 2022.

[19] Hongye Guo. Earnings extrapolation and predictable stock market returns. *The Review of Financial Studies*, 38(6):1730–1782, 2025.

[20] Bryan T Kelly, Semyon Malamud, Mohammad Pourmohammadi, and Fabio Trojani. Universal portfolio shrinkage. Technical report, National Bureau of Economic Research, 2024.

[21] Rishi K. Narang. *Inside the Black Box: A Simple Guide to Quantitative and High-Frequency Trading*. Wiley, Hoboken, NJ, 2nd edition, 2013.

[22] Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ, 2018.

[23] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020.

[24] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An AI-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020.

[25] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. Finrl: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*, pages 1–9, 2021.

[26] Andreas Neuhierl, Otto Randl, Christoph Reschenhofer, and Josef Zechner. Timing the factor zoo. *Available at SSRN*, 4376898, 2023.

[27] Bryan T. Kelly and Dacheng Xiu. Financial machine learning. *Foundations and Trends in Finance*, 13(3–4):205–363, 2023.

[28] Dong Lou and Christopher Polk. Comomentum: Inferring arbitrage activity from return correlations. 2013.

[29] Huafeng Chen, Shaojun Chen, Zhuo Chen, and Feng Li. Empirical investigation of an equity pairs trading strategy. *Management Science*, 65(1):370–389, 2019.

[30] Lauren Cohen, Yiwen Lu, and Quoc H Nguyen. Mimicking finance. Technical report, National Bureau of Economic Research, 2026.

[31] Steven L Heston, Christopher S Jones, Mehdi Khorram, Shuaiqi Li, and Haitao Mo. Option momentum. *The Journal of Finance*, 78(6):3141–3192, 2023.

[32] Lasse Heje Pedersen. *Efficiently Inefficient: How Smart Money Invests and Market Prices Are Determined*. Princeton University Press, Princeton, NJ, 2015.

[33] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. The SWE-bench illusion: When state-of-the-art LLMs remember instead of reason. In *Advances in Neural Information Processing Systems 39 Datasets and Benchmarks Track*, 2025.

[34] Shachar Kaufman, Saharon Rosset, and Claudia Perlich. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012.

[35] Stephen J. Brown, William N. Goetzmann, Roger G. Ibbotson, and Stephen A. Ross. Survivorship bias in performance studies. *The Review of Financial Studies*, 5(4):553–580, 1992.

[36] Richard G. Sloan. Do stock prices fully reflect information in accruals and cash flows about future earnings? *The Accounting Review*, 71(3):289–315, 1996.

[37] André F. Perold. The implementation shortfall: Paper versus reality. *The Journal of Portfolio Management*, 14(3):4–9, 1988.

[38] Robert Novy-Marx and Mihail Velikov. A taxonomy of anomalies and their trading costs. *The Review of Financial Studies*, 29(1):104–147, 2016.

[39] OpenAI. Codex CLI is a coding agent from OpenAI that runs locally on your computer. `https://github.com/openai/codex`, 2026.

[40] Opencode. `https://github.com/anomalyco/opencode/`, 2026. Accessed: 2026-05-02.

[41] Sina Ehsani and Juhani T Linnainmaa. Factor momentum and the momentum factor. *The Journal of Finance*, 77(3):1877–1919, 2022.

[42] Kent Daniel and Tobias J Moskowitz. Momentum crashes. *Journal of Financial economics*, 122(2):221–247, 2016.

[43] Lin William Cong, Guanhao Feng, Jingyu He, and Xin He. Growing the efficient frontier on panel trees. *Journal of Financial Economics*, 167:104024, 2025.

[44] Robert Novy-Marx. Fundamentally, momentum is fundamental momentum. Technical report, National Bureau of Economic Research, 2015.

[45] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[46] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 654–665, 2014.

[47] Frank Tip, Jonathan Bell, and Max Schaefer. LLMorpheus: Mutation testing using large language models. *IEEE Transactions on Software Engineering*, 51(6):1645–1665, 2025.

[48] Zhaolu Kang, Junhao Gong, Wenqing Hu, Shuo Yin, Kehan Jiang, Zhicheng Fang, Yingjie He, Chunlei Meng, Rong Fu, Dongyang Chen, Leqi Zheng, Eric Hanchen Jiang, Yunfei Feng, Yitong Leng, Junfan Zhu, Xiaoyou Chen, Xi Yang, and Richeng Xuan. Quanteval: A benchmark for financial quantitative tasks in large language models, 2026.

[49] Haohang Li, Yupeng Cao, Yangyang Yu, Shashidhar Reddy Javaji, Zhiyang Deng, Yueru He, Yuechen Jiang, Zining Zhu, K.p. Subbalakshmi, Jimin Huang, Lingfei Qian, Xueqing Peng, Jordan W. Suchow, and Qianqian Xie. INVESTORBENCH: A benchmark for financial decision-making tasks with LLM-based agent. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2509–2525, Vienna, Austria, July 2025. Association for Computational Linguistics.

[50] Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, et al. Finben: A holistic financial benchmark for large language models. *Advances in Neural Information Processing Systems*, 37:95716–95743, 2024.

[51] Alexey Khoroshilov, Alexey Chernysh, Orkhan Ekhtibarov, Nini Kamkia, and Dmitry Zmitrovich. Quantcode-bench: A benchmark for evaluating the ability of large language models to generate executable algorithmic trading strategies, 2026.

[52] Yuchen Cao, Hanlin Zhang, Jacky Wai Keung, Yang Chen, and Linqi Song. Systradebench: An iterative build-test-patch benchmark for strategy-to-code trading systems with drift-aware diagnostics, 2026.

[53] Abhay Srivastava, Sam Jung, and Spencer Mateega. Market-bench: Evaluating large language models on introductory quantitative trading and market dynamics, 2026.

[54] Reddit. `https://www.reddit.com/`, 2026. Accessed: 2026-05-02.

[55] Tradingview. `https://www.tradingview.com/ideas/`, 2026. Accessed: 2026-05-02.

[56] Pengcheng Chen, Ning Yan, Zihan Zhao, Yixiao Lin, Huaibo Chen, Yue Hu, Qinbo Bai, Xiang Li, and Masood S Mortazavi. Deep-reproducer: From paper understanding to code generation. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*.

[57] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[58] Clifford S Asness, Tobias J Moskowitz, and Lasse Heje Pedersen. Value and momentum everywhere. *The journal of finance*, 68(3):929–985, 2013.

[59] Tobias J Moskowitz, Yao Hua Ooi, and Lasse Heje Pedersen. Time series momentum. *Journal of financial economics*, 104(2):228–250, 2012.

[60] Jorge Guijarro-Ordonez, Markus Pelger, and Greg Zanotti. Deep learning statistical arbitrage. *Management Science*, 2025.

[61] Mamdouh Medhat and Maik Schmeling. Short-term momentum. *The Review of Financial Studies*, 35(3):1480–1526, 2022.

[62] Hossein Rad, Rand Kwong Yew Low, and Robert Faff. The profitability of pairs trading strategies: distance, cointegration and copula methods. *Quantitative finance*, 16(10):1541–1558, 2016.

[63] Sergio M Focardi, Frank J Fabozzi, and Ivan K Mitov. A new approach to statistical arbitrage: Strategies based on dynamic factor models of prices and their performance. *Journal of Banking & Finance*, 65:134–155, 2016.

[64] Feifei Wang and Xuemin Sterling Yan. Downside risk and the performance of volatility-managed portfolios. *Journal of banking & finance*, 131:106198, 2021.

[65] Dion Bongaerts, Xiaowei Kang, and Mathijs van Dijk. Conditional volatility targeting. *Financial Analysts Journal*, 76(4):54–71, 2020.

[66] Alan Moreira and Tyler Muir. Volatility-managed portfolios. *The Journal of Finance*, 72(4):1611–1644, 2017.

[67] Andrea Frazzini and Lasse Heje Pedersen. Betting against beta. *Journal of financial economics*, 111(1):1–25, 2014.

[68] Yong Chen, Zhi Da, and Dayong Huang. Short selling efficiency. *Journal of Financial Economics*, 145(2):387–408, 2022.

[69] Harrison G Hong, Weikai Li, Sophie X Ni, Jose A Scheinkman, and Philip Yan. Days to cover and stock returns, 2015.

[70] David O Lucca and Emanuel Moench. The pre-fomc announcement drift. *The Journal of finance*, 70(1):329–371, 2015.

[71] Pavel Savor and Mungo Wilson. Asset pricing: A tale of two days. *Journal of Financial Economics*, 113(2):171–201, 2014.

[72] Serhiy Kozak, Stefan Nagel, and Shrihari Santosh. Shrinking the cross-section. *Journal of Financial Economics*, 135(2):271–292, 2020.

[73] Valentin Haddad, Serhiy Kozak, and Shrihari Santosh. Factor timing. *The Review of Financial Studies*, 33(5):1980–2018, 2020.

[74] David E Rapach, Jack K Strauss, and Guofu Zhou. Out-of-sample equity premium prediction: Combination forecasts and links to the real economy. *The Review of Financial Studies*, 23(2):821–862, 2010.

[75] Edwin J. Elton, Martin J. Gruber, and Christopher R. Blake. Survivorship bias and mutual fund performance. *The Review of Financial Studies*, 9(4):1097–1120, 1996.

[76] James Reason. *Human Error*. Cambridge University Press, 1990.

[77] Donald A. Norman. Cognitive engineering. In Donald A. Norman and Stephen W. Draper, editors, *User Centered System Design: New Perspectives on Human–Computer Interaction*, pages 31–61. Lawrence Erlbaum Associates, 1986.

[78] Andrew J. Ko and Brad A. Myers. A framework and methodology for studying the causes of software errors in programming systems. *Journal of Visual Languages and Computing*, 16(1–2):41–84, 2005.

[79] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[80] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. Agentrx: Diagnosing ai agent failures from execution trajectories. *arXiv preprint arXiv:2602.02475*, 2026.

[81] Ivo Welch and Amit Goyal. A comprehensive look at the empirical performance of equity premium prediction. *The Review of Financial Studies*, 21(4):1455–1508, 2008.

[82] Kenneth R. French. Kenneth r. french data library. `https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/data_library.html`. Accessed: 2026-05-06.

[83] Theis Ingerslev Jensen, Bryan T. Kelly, and Lasse Heje Pedersen. Global factor data: Data download. `https://jkpfactors.com/data`. Accessed: 2026-05-06.

A Appendix Roadmap

Appendix B Related Work
Appendix C Paper Collection
C.1 Category Summary
C.2 Selected Finance Paper

Appendix D Paper Reproduction Task Example
D.1 Type-A Property
D.2 Type-B Property
D.3 User Instruction
D.3.1 Instruction Format
D.3.2 User Instruction Example
D.4 Checker

Appendix E Experimental Setup Details
E.1 Evaluated LLMs, Agents, and Repeated Attempts
E.2 Agent Input and Execution Environment
E.3 Experiment Platform

Appendix F Additional Property-Level Discrimination Results

Appendix G Root Cause Annotation Process

Appendix H Task Card Index

Appendix I Limitations and Future Work

Appendix J Licenses and Terms for Existing Assets Used in StrategyCodeEval

B Related Work

Finance and strategy-related benchmarks. Several benchmarks evaluate LLMs on financial question answering [48], market prediction [49, 50], and trading decisions [25]. Others explore strategy code generation, including QuantCodeBench [51], SysTradeBench [52], and MarketBench [53]. However, these benchmarks typically rely on short and concrete natural-language descriptions of relatively simple strategies, or on brief ideas collected from online communities such as Reddit [54] and TradingView [55]. In contrast, strategies described in academic finance papers are often substantially more complex, involving multi-asset cross-sectional ranking, portfolio construction, factor modeling, risk models, and implementation choices that interact across the multi-stage strategy pipeline.

Moreover, existing finance benchmarks generally lack fine-grained evaluation criteria, relying primarily on end-to-end trading metrics such as the Sharpe ratio. In contrast, QuantCode-Eval introduces a comprehensive property-based evaluation framework for assessing whether LLMs can faithfully reproduce strategies from real finance papers. It evaluates not only richer and more multidimensional strategy logic, but also realistic and robust backtesting implementations.

Research reproduction benchmarks. With the emergence of LLMs as research assistants [9], a growing number of benchmarks have been proposed to evaluate research-oriented code generation [56] and paper reproduction [8]. In particular, MLE-Bench [12] and MLGym [11] focus on machine learning engineering and ML paper replication. However, these benchmarks do not address the reproduction of quantitative finance strategies, a highly domain-specific setting in which both implementation and evaluation require substantial financial expertise. Determining whether a reproduced strategy is correct often depends on nuanced criteria, such as portfolio construction rules, factor definitions, risk controls, and backtesting assumptions, rather than generic code execution alone.

General coding benchmarks. General code generation benchmarks, such as LiveCodeBench [6], HumanEval [4], MBPP [5], and BigCodeBench [3], assess general LLM programming ability. Software engineering benchmarks such as SWE-bench [57] instead focus on resolving real-world GitHub issues. Neither line of work evaluates whether LLMs can reproduce quantitative strategies.

C Paper Collection

C.1 Category Summary

Table 4 summarizes the category-level composition of QuantCode-Eval. The collection is intentionally balanced across six strategy families, with five samples in each category, so that the benchmark does not overfit to a single category. The year ranges also show that the benchmark mixes classic foundational papers with recent work from 2023–2026, while the category descriptions highlight the methodological breadth of the suite, spanning cross-sectional return continuation, statistical arbitrage, characteristic-driven selection, volatility-based position scaling, event-driven trading, and portfolio allocation.

C.2 Selected Finance Papers

Table 5 lists the concrete papers selected in each category. The benchmark mostly draws from major finance venues (including The Journal of Finance, The Review of Financial Studies, Journal of Financial Economics, Management Science, and Journal of Machine Learning Research), combining highly cited canonical papers with newer, lower-citation studies that reflect emerging research directions. Classic papers contribute stable and influential baselines, whereas recent papers and preprints introduce more novel modeling choices.

Table 5: Papers used in QuantCode-Eval (citation counted by May, 2026).

CID	Year	Paper Title	Publication Outlet	Citation
C1	2013	Value and Momentum Everywhere [58]	The Journal of Finance	3918

Continued on next page

CID	Year	Paper Title	Publication Outlet	Citation
C1	2022	Comomentum: Inferring Arbitrage Activity from Return Correlations [28]	The Review of Financial Studies	105
C1	2016	Momentum Crashes [42]	Journal of Financial Economics	1690
C1	2015	Fundamentally, Momentum Is Fundamental Momentum [44]	Journal of Financial Economics	133
C1	2012	Time Series Momentum [59]	Journal of Financial Economics	2096
C2	2025	Deep Learning Statistical Arbitrage [60]	Management Science	86
C2	2022	Short-term Momentum [61]	The Review of Financial Studies	90
C2	2019	Empirical Investigation of an Equity Pairs Trading Strategy [29]	Management Science	164
C2	2016	The Profitability of Pairs Trading Strategies: Distance, Cointegration and Copula Methods [62]	Quantitative Finance	268
C2	2016	A New Approach to Statistical Arbitrage: Strategies Based on Dynamic Factor Models of Prices and Their Performance [63]	Journal of Banking & Finance	38
C3	2026	Mimicking Finance: Predicting Mutual Fund Manager Behavior Using Deep Learning [30]	National Bureau of Economic Research	1
C3	2025	Growing the Efficient Frontier on Panel Trees [43]	Journal of Financial Economics	65
C3	2024	Timing the Factor Zoo [26]	SSRN	25
C3	2023	Factor Momentum and the Momentum Factor [41]	The Review of Financial Studies	339
C3	2020	Empirical Asset Pricing via Machine Learning [23]	The Review of Financial Studies	3465
C4	2023	Option Momentum [31]	The Journal of Finance	73
C4	2021	Downside Risk and the Performance of Volatility-Managed Portfolios [64]	Journal of Banking & Finance	65
C4	2020	Conditional Volatility Targeting [65]	Financial Analysts Journal	40
C4	2017	Volatility-Managed Portfolios [66]	The Journal of Finance	749
C4	2014	Betting Against Beta [67]	Journal of Financial Economics	3237
C5	2025	Earnings Extrapolation and Predictable Stock Market Returns [19]	The Review of Financial Studies	15
C5	2022	Short Selling Efficiency [68]	Journal of Financial Economics	61
C5	2016	Days to Cover and Stock Returns [69]	SSRN	68
C5	2015	The Pre-FOMC Announcement Drift [70]	The Journal of Finance	902
C5	2014	Asset Pricing: A Tale of Two Days [71]	Journal of Financial Economics	457
C6	2024	Two Is Better Than One: Regularized Shrinkage of Large Minimum Variance Portfolios [13]	Journal of Machine Learning Research	17
C6	2023	Universal Portfolio Shrinkage [20]	National Bureau of Economic Research	17
C6	2020	Shrinking the Cross-Section [72]	Journal of Financial Economics	973
C6	2020	Factor Timing [73]	The Review of Financial Studies	171
C6	2010	Out-of-Sample Equity Premium Prediction: Combination Forecasts and Links to the Real Economy [74]	The Review of Financial Studies	2000

D Paper Reproduction Task Example

In this section, we take *Short-term Momentum* [18] as a reproduction task example. We first present the Type-A properties, which capture domain-level constraints shared across different quantitative trading strategy papers. We then introduce the Type-B properties, which are paper-specific requirements extracted from *Short-term Momentum*. Next, we show the user instructions, which explicitly specify ambiguous details of each properties, forming a joint input together with the paper. Finally, we present the checkers for each property. Here, we only include several representative examples of the checkers.

Introduction of *Short-term Momentum*. This sample implements a monthly long-short equity strategy that runs the paper's two core sleeves, STMOM and STREV*, in parallel on the same CRSP stock-month panel. STMOM denotes the short-term momentum sleeve, which exploits the

Table 4: The summary of categories in QuantCode-Eval.

ID	Category	Count	Year Range	Characteristics of This Kind of Strategies
C1	Momentum	5	2012–2022	Long-short strategies that exploit persistence in past asset returns.
C2	Mean Reversion	5	2016–2022	Pair trading and statistical arbitrage strategies built on prices reverting after deviations from equilibrium.
C3	Factor	5	2020–2026	Cross-sectional factor selection and factor-timing strategies driven by observable characteristics.
C4	Volatility	5	2014–2023	Position management and risk-hedging strategies based on volatility signals.
C5	Event-Driven	5	2014–2025	Strategies triggered by specific events such as earnings announcements, analyst behavior, or short-selling signals.
C6	Systematic Allocation	5	2010–2024	Portfolio-weight optimization and dynamic asset-allocation strategies.

continuation of very recent winners relative to losers within the high-turnover segment of the cross section. STREV* denotes the short-term reversal sleeve, which exploits the rebound of very recent losers relative to winners within the low-turnover segment of the cross section. The eligible universe is limited to ordinary common shares (share codes 10 and 11) listed on NYSE, AMEX, or NASDAQ, with financial firms excluded. For each month, the strategy uses the previous month's CRSP holding-period return and share turnover as its two sorting variables, where turnover is constructed as monthly trading volume scaled by shares outstanding and NASDAQ volume is adjusted using the paper's period-specific correction factors.

Portfolio formation follows the conditional sorting procedure in the paper. Stocks are first assigned to return deciles using NYSE breakpoints, and then, within each return decile, they are assigned to turnover deciles using NYSE breakpoints again. STMOM trades only within the highest-turnover bucket, taking a value-weighted long position in past return winners and a value-weighted short position in past return losers. STREV* trades only within the lowest-turnover bucket, taking the opposite return-side exposure: long past losers and short past winners. Both sleeves are formed at the end of month $t-1$, held during month t, and rebalanced every month. The final sample strategy combines the two completed sleeve return series at the return level using a fixed 50/50 weight, yielding a single monthly long–short portfolio used for evaluation.

D.1 Type-A Property

Type-A properties are correctness constraints that any valid and backtestable quantitative strategy must satisfy. Thus, these properties are shared across the benchmark because they come from pipeline stages common to most strategies.

Derivation: six-stage strategy pipeline Following §2, we view a quantitative strategy implementation as a six-stage backtesting pipeline: 1) data loading, 2) feature engineering, 3) model fitting, 4) signal generation, 5) portfolio construction, and 6) execution. For each stage, we identify the primary stage-level integrity failure modes: violations that would make a quantitative backtest invalid regardless of the specific paper being reproduced. This yields nine domain-level integrity properties from six stages. Each property is the canonical instance of its stage's primary failure mode in the published backtesting-methodology literature, cited inline in the per-property entries.

A1 `no_backfill` (Stages S1, S2). Missing values in raw market or fundamental series must not be filled using observations from later timestamps; any imputation used at time t must be based only on information available no later than t [34].

A2 `no_global_normalize` (Stage S2). Feature scaling or normalization must not be fit on the full sample when the result is used for historical decisions. Scaling statistics must be computed from

information available at the decision time, such as rolling, expanding, or same-date cross-sectional statistics [34].

A3 `no_future_data_access` (Stages S1, S2). Every feature, label proxy, or intermediate state used for a decision at time t must be measurable with respect to observations available by t; it must not depend on $\{x_\tau : \tau > t\}$ [34].

A4 `causal_consistency` (Stages S4, S5). The implementation must preserve the causal order of the backtesting pipeline: data loading, feature engineering, model fitting when applicable, signal generation, portfolio construction, and execution. Earlier-stage quantities must not depend on outputs or state produced only by later stages [22].

A5 `no_survivorship_bias` (Stage S1). The tradable universe at time t must be constructed point-in-time, using the constituents that were eligible at t rather than a survivor-only universe observed after the sample period [75, 35].

A6 `train_test_temporal_order` (Stage S3). Model parameters, hyperparameters, and model-selection decisions must be learned using only data preceding the evaluation period to which they are applied; training or validation data must not contaminate later test periods [22].

A7 `data_frequency_alignment` (Stage S1). When data sources with different frequencies or reporting conventions are combined, each observation must enter the backtest only after it would have become observable. Lower-frequency fundamentals or event records must be aligned by release or availability date rather than by period-end date [36].

A8 `transaction_timing` (Stage S6). Portfolio decisions must be evaluated using an implementable execution convention. The fill price and timestamp must be consistent with the declared execution rule, rather than mixing close, open, or same-period prices in ways that could not have been traded in real time [37].

A9 `signal_delay_test` (Stage S4). A signal computed at time t must be executed only after the minimum delay implied by the strategy's data frequency and execution convention. Same-period execution is disallowed unless the information used to form the signal is observable before the declared execution time [38].

A10 `e2e_metric_consistency` (Stage S6). The final long–short performance should remain close to the paper benchmark (Winner−Loser in TO D10: $+1.37\%$, $t = 4.74$) within the predefined tolerance. This is the aggregate check: even small mistakes in universe filtering, conditional sorting, weighting, or timing will compound into a material deviation in the reported end-to-end return.

D.2 Type-B Property

Type-B properties are paper-specific and must capture the critical formulas and design choices of each strategy. Thus, we take *Short-term Momentum* [18] as a case study to illustrate representative Type-B properties in detail. The extraction of Type-B is evidence-driven. For each Type-B property, we attach a source anchor from the paper whenever possible, including section text, figure descriptions, and footnotes.

B1 `monthly_return_calc` (Stages S1, S2). Use CRSP monthly holding-period return directly, because it already includes dividends and distributions. Recomputing return from prices alone often drops important adjustments. The signal is specifically the previous month's 1-month return $r_{1,0}$, and the same definition is shared by both STMOM and STREV* sleeves. *Source: paper §1.1, "asset pricing tests relate 1-month returns to the previous month's return ($r_{1,0}$)"; implementation spec further fixes this to the CRSP monthly ret field.*

B2 `turnover_calc` (Stage S2). Turnover is $\mathrm{TO}_{1,0} = \sum_d \mathrm{vol}_d / \mathrm{shrout}$: sum daily volume over the month, then divide by shares outstanding. Monthly average volume is not the same quantity. The numerator is the month's total trading volume, not an average, and the denominator is shares

outstanding. This same turnover definition is shared by both sleeves. *Source: paper §1.1, "we deflate the total volume of trades by the number of shares outstanding to obtain share turnover (*$\mathrm{TO}_{1,0}$*)".*

B3 `nasdaq_volume_adjustment` (Stage S2). Before turnover is computed, NASDAQ volume must be adjusted by period: $\div 2.0$ before 2001-02, $\div 1.8$ from 2001-02 to 2001-12, $\div 1.6$ from 2002-01 to 2003-12, and no adjustment after 2004-01. This correction applies only to NASDAQ stocks (exchcd $= 3$) and must be done before monthly turnover is aggregated; otherwise the turnover ranking is systematically distorted. *Source: paper footnote 6, "Prior to February 2001, we divide NASDAQ volume by 2.0 ... From January 2004 and onward ... we apply no adjustment."*

B4 `nyse_breakpoints_return` (Stage S2). Return decile cutoffs must be estimated from NYSE stocks only (exchcd$= 1$), then applied to all eligible stocks. Using all exchanges to compute cutoffs is not consistent with the design. The breakpoint universe and the application universe are intentionally different: NYSE names determine the thresholds, but all eligible names are assigned to deciles using those thresholds. *Source: paper §1.2, "conditional decile sorts based on NYSE breakpoints".*

B5 `nyse_breakpoints_turnover` (Stage S2). Within each return decile, turnover cutoffs must be computed from NYSE stocks in that same return decile. A single global turnover cutoff over the full universe is incorrect. This is a nested breakpoint rule: after the first sort on return, turnover breakpoints are recomputed separately inside each return bin using only NYSE stocks in that bin. *Source: paper §1.2, "first on* $r_{1,0}$ *and then on* $\mathrm{TO}_{1,0}$ *within* $r_{1,0}$ *deciles".*

B6 `conditional_sort_order` (Stages S2, S4). Sorting must be sequential: first by return decile, then by turnover decile within each return bin. Independent two-way sorting over the full universe does not produce equivalent portfolios. This is one of the most important design details in the paper, because conditional sorting changes the set of stocks in each corner portfolio. *Source: paper §1.2, "We use conditional decile sorts based on NYSE breakpoints, first on* $r_{1,0}$ *and then on* $\mathrm{TO}_{1,0}$ *within* $r_{1,0}$ *deciles".*

B7 `stmom_turnover_selection` (Stage S4). The STMOM sleeve uses only TO D10 (highest-turnover stocks). Including other turnover groups violates the intended construction. So even if a stock is a winner or loser on past return, it is not eligible for the STMOM sleeve unless it also falls into the highest-turnover decile. *Source: paper §1.2, Figure 1, "short-term momentum trades only in the highest-turnover decile".*

B8 `strev_turnover_selection` (Stage S4). The STREV* sleeve uses only TO D1 (lowest-turnover stocks). Including other turnover groups violates the intended construction. So STREV* is restricted to the opposite corner of the turnover dimension and must not trade outside that low-turnover bucket. *Source: paper §1.2, Figure 1, "short-term reversal* trades only in the lowest-turnover decile".*

B9 `stmom_long_short_signal` (Stage S4). STMOM must be formed as: Long $=$ Ret D10 $\cap$ TO D10, Short $=$ Ret D1 $\cap$ TO D10. This is the winner-minus-loser momentum direction. Reversing the long and short legs would flip the sign of the strategy and fail to reproduce the reported STMOM sleeve. *Source: paper §1.2, Figure 1, "short-term momentum strategy, which buys the previous month's winners and shorts the losers within the highest turnover decile".*

B10 `strev_long_short_signal` (Stage S4). STREV* must be formed as: Long $=$ Ret D1 $\cap$ TO D1, Short $=$ Ret D10 $\cap$ TO D1. This is the short-term reversal direction, namely loser-minus-winner inside the lowest-turnover decile. The paper describes STREV* through its negative WML return; operationally, that implies going long losers and short winners in the TO D1 bucket. *Source: paper §1.2, Figure 1; anchored by the paper's STREV* description and the reported negative WML sign in the low-turnover corner portfolio.*

B11 `value_weighted_portfolios` (Stage S5). Both STMOM and STREV* long and short legs must be value-weighted by market cap. Equal weighting is a common but incorrect implementation. The value-weighting requirement applies to both sleeves and to both sides of each long–short portfolio, not just to the final combination stage. *Source: paper §1.2, Figure 1, "The portfolios underlying all three strategies are value weighted and rebalanced at the end of each month."*

B12 `monthly_rebalance` (Stages S5, S6). Both sleeves must be rebalanced every month-end. Skipped months or asynchronous rebalancing violate the strategy definition. The timing requirement is both monthly and end-of-month; this is not a generic "rebalance sometimes" rule. *Source: paper §1.2, Figure 1, "The portfolios underlying all three strategies are value weighted and rebalanced at the end of each month."*

B13 `dual_sleeve_architecture` (Stages S4, S5). STMOM and STREV* should be implemented as two separate sleeves on the same panel, each producing its own long–short return before combination. A correct plan_combo implementation therefore needs two distinct signal-generation paths that operate on the same stock-month panel and decile assignments. *Source: paper Figure 1 and Table 1, which present STMOM and STREV* as separate corner-portfolio strategies; plan_combo uses both sleeves together.*

B14 `equal_weight_blending` (Stage S5). The final blend must be equal-weighted: $r_{\mathrm{combo},t} = 0.5\, r_{\mathrm{stmom},t} + 0.5\, r_{\mathrm{strev},t}$. Using a different mixing ratio breaks the specification. This 50/50 blend is a plan_combo implementation rule rather than a paper-authored portfolio in the original article. *Source: sample spec combination_rule, "$r_\mathrm{combo}, \mathrm{t} = 0.5 * r_\mathrm{stmom}, \mathrm{t} + 0.5 * r_\mathrm{strev}, \mathrm{t}$" (spec-derived, not directly stated in the paper).*

B15 `return_level_combination` (Stage S5). The two sleeves must be combined at return level, after each sleeve has its own monthly long–short return. Signals or positions must not be merged earlier, otherwise the resulting portfolio is not the same as averaging two completed sleeve returns. *Source: sample spec combination_rule method, "equal_weight_monthly_returns" (spec-derived implementation detail).*

B16 `universe_share_codes` (Stage S1). The stock universe must include only CRSP share codes 10 and 11 (common shares). Other share classes should be excluded, and this filter applies before any sorting or weighting. *Source: paper §1.1, "We keep only ordinary common shares (share codes 10 or 11)."*

B17 `universe_exclude_financials` (Stage S1). Financial firms (SIC 6000–6999) must be excluded when constructing the monthly universe. This exclusion is part of the paper's sample design rather than an optional robustness choice. *Source: paper §1.1, "Following Hou, Xue, and Zhang (2015, 2020), we exclude financial firms".*

B18 `universe_exchanges` (Stage S1). Eligible stocks are restricted to NYSE, AMEX, and NASDAQ (exchcd $\in \{1, 2, 3\}$). Other exchanges are outside scope, and this same exchange filter is shared by both sleeves. *Source: paper §1.1, "Our main sample is all NYSE/AMEX/NASDAQ stocks".*

B19 `formation_holding_period_1m` (Stages S2, S6). Both formation window and holding window must be exactly one calendar month for both sleeves. Other horizons do not match the target design, because the paper defines the signal from the previous month's return and then holds over the next month. *Source: paper §1.1, "Our asset pricing tests relate 1-month returns to the previous month's return ($r_{1,0}$)".*

B20 `simultaneous_execution` (Stage S6). STMOM and STREV* must be formed on the same month-end and held over the same month t. Alternating or staggered execution is not allowed. In the combo implementation, both sleeves run in parallel on the same monthly schedule and only then get combined. *Source: paper Figure 1 for the common monthly rebalancing schedule; plan_combo spec requires both sleeves to execute on the same date grid.*

D.3 User Instruction

This section presents the user instruction together with the financial paper as a joint input. As described in §3.1, the instruction is an agent-facing task contract: it specifies importable function signatures, input-output schemas, timestamp semantics when they are part of the task definition, and expert refinements for paper details that are under-specified as software requirements. The common execution boilerplate shared by all tasks (e.g., command-line invocation and required JSON metrics) is omitted here; the example is restricted to the sample-specific contract supplied to evaluated agents.

D.3.1 Instruction Format

Table 6 summarizes the fields that appear in the sample-specific instruction excerpt. The format is intentionally narrow: it defines the public interface and the expert clarifications needed to make a paper-level strategy executable, while leaving implementation choices to the agent wherever the paper and task contract do not constrain them.

Table 6: Structure of the Sample Instruction shown to evaluated agents.

Component	What is specified	Role in the task
Required functions	Top-level Python functions that the submitted `strategy.py` must expose.	Provides stable entry points for execution and component-level evaluation.
Typed IO shape	Function arguments, return type, required DataFrame columns, and row granularity.	Fixes the public software interface without prescribing a particular implementation.
Temporal anchors	Timestamp meaning for outputs whose dates are otherwise ambiguous.	Ensures that signals, weights, and realized returns are aligned consistently with the paper's trading convention.
Expert refinements	Domain-expert clarifications for paper details that must be resolved before the task can be implemented.	Makes the reproduction target deterministic while preserving the paper as the primary methodological source.

D.3.2 User Instruction Example: Short-Term Momentum and Reversal

The boxed excerpt below is drawn from the short-term momentum sample of Medhat and Schmeling [18]. The task is a representative multi-stage reproduction problem: the strategy filters a CRSP equity universe, computes monthly return and turnover signals, assigns conditional NYSE-breakpoint deciles, constructs two long-short sleeves corresponding to the paper's high-turnover momentum and low-turnover reversal patterns, and evaluates monthly portfolio returns. The excerpt uses the instruction components summarized in Table 6.

Agent-facing User Instruction excerpt (1/4): Required functions

```
# Interface Contract:  STMOM + STREV* - plan_combo
```

Required Functions

All required functions must be importable as top-level module functions. Parameter names should match the input DataFrame names shown below; a trailing `_df` suffix is accepted for pipeline inputs. The functions define observable pipeline boundaries for evaluation; they do not prescribe a particular internal implementation.

#	Function	Key output columns
1	`compute_universe(crsp)`	`eligible`
2	`compute_turnover(crsp)`	`turnover`
3	`compute_decile_assignments(panel_df)`	`return_decile`, `turnover_decile`
4	`compute_stm_trade_signal(decile_df)`	`signal`
5	`compute_strev_trade_signal(decile_df)`	`signal`
6	`compute_portfolio_weights(signal_df, signal_col)`	`mktcap`, `weight`
7	`compute_sleeve_portfolio_returns(weights_df)`	`portfolio_ret`
8	`compute_combined_portfolio_returns(decile_df)`	`portfolio_ret_stmom`, `portfolio_ret_strev`, `portfolio_ret`

Agent-facing User Instruction excerpt (2/4): IO shape

IO Shape

The IO contract specifies schemas that downstream code relies on. It does not specify hidden tests or reveal the checker implementation. Algorithms, parameters, and formulas not fixed by this contract must be derived from the paper and the refinements below.

Function	Input-output contract
`compute_universe (crsp) -> pd.DataFrame`	**Input:** `crsp` with columns `[permno, date, ret, prc, shrout, exchcd, shrcd, siccd, ...]`. **Output:** DataFrame preserving stock-month rows and adding `eligible`; one row per `(permno, date)` observation.
`compute_turnover (crsp) -> pd.DataFrame`	**Input:** `crsp` with `[permno, date, vol, shrout, exchcd, ...]`. **Output:** DataFrame preserving stock-month rows and adding `turnover`.
`compute_decile_assignments (panel_df) -> pd.DataFrame`	**Input:** monthly panel with `[permno, date, ret, turnover, exchcd, eligible, market_cap, ...]`. **Output:** DataFrame with `[permno, date, return_decile, turnover_decile]`. Deciles are labeled from 1 to 10.
`compute_stm_trade_signal (decile_df) -> pd.DataFrame`	**Input:** `decile_df` with `[permno, date, return_decile, turnover_decile, ...]`. **Output:** DataFrame preserving stock-month rows and adding `signal`, whose values are `long`, `short`, or `neutral`.
`compute_strev_trade_signal (decile_df) -> pd.DataFrame`	**Input:** same shape as `compute_stm_trade_signal`. **Output:** DataFrame preserving stock-month rows and adding the same `signal` column for the reversal sleeve.
`compute_portfolio_weights (signal_df, signal_col=signal) -> pd.DataFrame`	**Input:** `signal_df` with `[permno, date, signal, market_cap, ret, ...]`. **Output:** DataFrame with `[permno, date, signal, mktcap, weight, ret]`. The `weight` column is signed, with positive values for long positions and negative values for short positions.
`compute_sleeve_ portfolio_returns (weights_df) -> pd.DataFrame`	**Input:** `weights_df` with `[permno, date, weight, ret, ...]`. **Output:** monthly DataFrame indexed or keyed by `date` with `[long_ret, short_ret, portfolio_ret]`. The output date is the realized holding month.
`compute_combined_ portfolio_returns (decile_df) -> pd.DataFrame`	**Input:** `decile_df` with the columns needed by both sleeve signal functions and weighting. **Output:** monthly DataFrame with `[portfolio_ret_stmom, portfolio_ret_strev, portfolio_ret]`; `portfolio_ret` is the combined monthly return.

Agent-facing User Instruction excerpt (3/4): Expert annotations

Expert Refinements

The following refinements resolve task-level choices that must be fixed before the paper can be implemented as a deterministic benchmark instance. They are included when the paper states the convention explicitly, when the selected benchmark variant must be disambiguated, or when a data convention must be made operational.

R1: Universe filter

Source: paper sample description and CRSP data convention. The tradable universe consists of ordinary common shares (`shrcd` $\in \{10, 11\}$) on NYSE/AMEX/NASDAQ (`exchcd` $\in \{1, 2, 3\}$), excluding financial firms (`siccd` $\in [6000, 6999]$). Construct the universe month by month from the provided panel; do not replace it with a survivor-only list.

R2: Turnover convention

Source: paper footnote on NASDAQ volume adjustment and CRSP data convention. The `turnover` output is the monthly share-turnover measure used by the paper. Apply the NASDAQ volume adjustment described in the paper's footnote before computing turnover. When using CRSP `shrout`, account for its reported unit convention when converting to share count.

R3: Conditional decile assignment

Source: paper description of conditional NYSE-breakpoint sorts. Return deciles are computed monthly from NYSE stocks and then assigned to all eligible stocks. Turnover deciles are then computed within each return decile, again using NYSE stocks as the breakpoint source. Do not replace this with a single pooled sort or two independent unconditional sorts.

Agent-facing User Instruction excerpt (4/4): Expert annotations

R4: Sleeve definitions

Source: paper Table 1 and benchmark variant selection. The sample implements both Table 1 sleeves. The short-term momentum sleeve uses the high-turnover side of the conditional sort. The short-term reversal sleeve uses the low-turnover side with the long-short direction reversed relative to the momentum sleeve. The two sleeve signal functions must remain separate top-level functions.

R5: Weighting and combination

Source: paper portfolio construction and benchmark variant selection. Within each long or short leg, positions are value-weighted using formation-date market capitalization. The combined series is formed at the return level as

$$r_{\mathrm{combo},t} = 0.5\, r_{\mathrm{STMOM},t} + 0.5\, r_{\mathrm{STREV},t}.$$

The 50/50 combination is a benchmark default for the selected dual-sleeve task; it is not a reported paper performance target.

R6: Holding period and rebalance

Source: paper monthly portfolio convention. The strategy forms portfolios from previous-month return and turnover signals and earns returns over the following monthly holding period. Rebalance at month-end. The output date in the return functions denotes the realized holding month, not the signal-observation month.

D.4 Checker

We select `A3`, `A4`, `B13`, and `B15` as representative examples to show the checker implementation.

Type-A checker: `A3 no_future_data_access`. This checker targets front-running through return alignment. Its test panel is constructed so that formation-month and holding-month long–short returns are very different: if the implementation correctly uses next-month realized returns, the first period's portfolio return is small; if it mistakenly reads the same row's formation-month return as the realized return, the first period becomes much larger and immediately reveals a look-ahead leak. As with A4, the checker also tolerates agents that follow a same-row signed-weight convention, provided the function itself does not internally shift data forward.

```
# A3 core logic: distinguish holding-month returns
# from formation-month returns.
result_golden = _normalize_result(fn(df_golden))

expected_p1 = 0.01 - (-0.01)   # correct holding-month long-short return
buggy_p1 = 0.50 - (-0.30)      # incorrect formation-month long-short return
actual_p1 = result_golden.iloc[0]["portfolio_ret"]

if abs(actual_p1 - expected_p1) < tol:
 return CheckResult(
  property_id=PROPERTY_ID,
  verdict="PASS",
  evidence="Holding-month returns correctly used"
 )

if abs(actual_p1 - buggy_p1) < tol:
 return CheckResult(
  property_id=PROPERTY_ID,
  verdict="FAIL",
  evidence="Formation-month returns used; future data leak detected"
 )
```

The value of A3 is that it isolates one concrete leakage channel that often appears even when the rest of the pipeline looks structurally correct. In practice, many candidate implementations do form the right long and short legs, but then accidentally attribute the contemporaneous signal-month return to that portfolio. The checker therefore acts as a sharply targeted anti-look-ahead test: it converts a subtle timestamp misalignment into a large, easy-to-audit difference in the first reported portfolio return.

Type-A checker: `A4 causal_consistency`**.** This checker targets the benchmark's most important domain guard: the strategy must use formation-month signals to determine next-month positions, rather than using holding-month information when the return is realized. The implementation is deliberately designed around a micro-dataset whose long/short signals flip across adjacent months. Under the correct convention, the first holding-period return must equal the return earned by the portfolio formed from the previous month's signals; under the buggy convention, the sign flips immediately because the checker would be reading the wrong month's labels. The checker also keeps a second acceptance path for agents that implement same-row signed-return aggregation, so it distinguishes genuine causal violations from harmless IO convention differences.

```
# A4 core logic: flip signals across months to separate
# formation-month and holding-month conventions.
result_golden = _normalize_result(fn(df_golden))

expected_p1 = 0.10 - (-0.05)   # Jan signals applied to Feb returns
buggy_p1 = -0.05 - 0.10        # Feb signals mistakenly applied to Feb returns
actual_p1 = result_golden.iloc[0]["portfolio_ret"]

if abs(actual_p1 - expected_p1) < tol:
 return CheckResult(
  property_id=PROPERTY_ID,
  verdict="PASS",
  evidence="Formation-month signals correctly used"
 )

if abs(actual_p1 - buggy_p1) < tol:
 return CheckResult(
  property_id=PROPERTY_ID,
  verdict="FAIL",
  evidence="Holding-month signals used instead of formation-month signals"
 )
```

The key to this design is not numerical complexity but discriminability. By constructing a panel where the signal direction reverses between consecutive months, the checker turns a hidden temporal bug into an observable sign change in the first portfolio return. In other words, the checker does not merely ask whether the output "looks reasonable"; it creates a local counterfactual in which the correct and incorrect implementations produce opposite answers.

Type-B checker: `B13 dual_sleeve_architecture`**.** This checker verifies that the combo strategy preserves the paper's two-sleeve design before any final blending occurs. It first checks for the existence of two top-level signal functions, `compute_stm_trade_signal` and `compute_strev_trade_signal`, and then feeds both functions a synthetic panel whose four corners correspond to the economically important quadrants of the conditional sort. The test is designed so that STMOM should go long high-return/high-turnover names, whereas STREV* should go long low-return/low-turnover names. The checker then confirms that the combined-return function exposes separate sleeve-return columns rather than only a single final series.

```
# B13 core logic: verify two independent sleeve functions
# and confirm they generate different corner signals.
df_stm = module.compute_stm_trade_signal(df.copy())
df_strev = module.compute_strev_trade_signal(df.copy())

stm_long = df_stm[
 (df_stm['return_decile'] == 10) &
 (df_stm['turnover_decile'] == 10)
]['signal'].iloc[0]

strev_long = df_strev[
 (df_strev['return_decile'] == 1) &
 (df_strev['turnover_decile'] == 1)
```

```
][’signal’].iloc[0]

if stm_long != ’long’ or strev_long != ’long’:
 return CheckResult(
  property_id="B13",
  verdict="FAIL",
  evidence="STMOM/STREV* sleeves do not produce distinct target signals"
 )
```

The strength of B13 is that it checks architectural faithfulness rather than only numeric correctness. In this benchmark, the combo strategy is not just a single long–short rule with a post-hoc label split; it is a composition of two economically distinct sleeves. B13 therefore guards against a common shortcut: defining the required function names but routing both of them through the same underlying logic, which would erase the paper's momentum-versus-reversal separation even if the final aggregate return still looks superficially valid.

Type-B checker: `B15 return_level_combination`**.** This checker is the best representative of the paper-specific combo logic because it verifies the exact place where the two sleeves are allowed to meet. It first drives the candidate strategy through the upstream universe, turnover, and decile pipeline so that the tested function receives a realistic stock-month panel. It then requires the output to expose three monthly series: `portfolio_ret_stmom`, `portfolio_ret_strev`, and the final `portfolio_ret`. The decisive test is whether the final series satisfies

$$r_{\mathrm{combo},t} = 0.5\, r_{\mathrm{STMOM},t} + 0.5\, r_{\mathrm{STREV},t}$$

month by month. If the implementation blends positions or signals before each sleeve return is computed, this identity will generally break even when the final output still appears plausible.

```
# B15 core logic: the final combined return must be the
# average of two already-computed sleeve returns.
required_cols = [
 "portfolio_ret_stmom",
 "portfolio_ret_strev",
 "portfolio_ret",
]

for i in range(len(result)):
 expected_combined = (
  0.5 * stmom_returns[i] + 0.5 * strev_returns[i]
 )
 actual_combined = combined_returns[i]

 if abs(actual_combined - expected_combined) > 0.01:
  return CheckResult(
   property_id="B15",
   verdict="FAIL",
   evidence=(
    f"Month {i}: combined={actual_combined:.4f}, "
    f"expected={expected_combined:.4f}"
   ),
   detail="May be blending positions instead of sleeve returns"
  )
```

This checker is representative because it tests the sample's defining expert refinement rather than a generic implementation preference. The benchmark does not only require that both STMOM and STREV* exist; it requires that each sleeve remain economically complete on its own, with its own long-short return, before the benchmark's fixed 50/50 combination is applied. That makes B15 a compact but high-value checker: it simultaneously checks interface completeness, architectural separation, and faithfulness to the task-level portfolio construction rule.

E Experimental Setup Details

This appendix gives the reproducibility details for the model evaluation in §4. The main text summarizes the evaluation setup. We specify the evaluated systems, repeated-attempt procedure, execution harness, resource limits, runtime environment, failure accounting, and archived artifacts used to produce the reported results.

E.1 Evaluated LLMs, Agents, and Repeated Attempts

We evaluate coding agents on the 30 benchmark tasks in QuantCode-Eval. Each task corresponds to a selected paper-strategy pair together with its user instruction and prepared data files. The evaluation includes two agent configurations. First, we evaluate closed-source frontier models in their native coding-agent scaffolds, using Claude Code for Claude models and the OpenAI Codex CLI for GPT models. Second, we evaluate all model backends under a common OpenCode scaffold, which holds the surrounding agent loop fixed while varying the LLM backend.

For each benchmark instance and evaluated LLM-agent combination, we run five independent attempts under the same prompt, data files, scaffold, model, and resource budget. Let c_i denote the number of successful submissions among the five attempts for benchmark instance i. Following the standard code generation convention, we estimate

$$\widehat{\text{pass@}k}_i = 1 - \frac{\binom{5-c_i}{k}}{\binom{5}{k}}, \qquad k \in \{1, 5\}.$$

With five attempts, this reduces to $\widehat{\text{pass@}1}_i = c_i/5$ and $\widehat{\text{pass@}5}_i = 1$ if $c_i > 0$ and 0 otherwise. Aggregate pass@1 and pass@5 are the averages of these task-level estimates over the 30 benchmark instances. The task-level pass@1 value is therefore an empirical estimate of single-attempt success probability, not a binary task label. All five attempts are run regardless of early successes. Attempts start from fresh workspaces and do not share generated code, logs, intermediate files, agent memory, or validation feedback. pass@5 measures repeated sampling under the same task specification rather than multi-stage repair of a previous submission. Each attempt produces at most one submitted `strategy.py`, which is then evaluated by the same harness and checker suite as all other attempts for that benchmark instance.

E.2 Agent Input and Execution Environment

Each attempt is initialized from the same reproduction task materials: the full paper text, the user instruction for the selected benchmark variant, the prepared local data files, and lightweight data descriptors listing file schemas and date ranges. The prompt exposes the working directory as `.` rather than as an absolute repository path, and instructs the agent to load only local data from `./data/`. Agents are not asked to install additional packages or retrieve external data during generation.

Runs are executed in fresh working directories. Data files are materialized as read-only files inside the run workspace, while the paper text and task instruction are copied into the workspace so that the agent does not need to access the benchmark repository. The shared virtual environment provides the pinned Python dependencies used by all attempts, but run-level files such as `strategy.py`, logs, generated artifacts, and agent state are not shared across attempts. The workspace contains the task-facing materials needed to write the strategy and excludes the evaluation artifacts used for grading. During strategy reproduction, agents receive no checker output, golden-reference output, or validation feedback.

The submitted artifact for an attempt is a single Python file, `strategy.py`. For backtrader-based tasks, the file must expose `MyStrategy(bt.Strategy)` as the entry point. For pandas-based tasks, it must expose a `main()` routine that runs the strategy and prints the required JSON metrics. The execution environment selects the execution route from the task metadata, runs the submitted strategy on the prepared data, records any reported metrics and trade log, and then invokes the task's property-checker suite on the submitted code.

E.3 Experiment Platform

All attempts use the same local execution environment, prepared data files, and pinned Python environment. LLM inference is performed through the corresponding remote model APIs, while the local host is responsible for workspace management, command execution, strategy backtesting, checker execution, and artifact logging.

Table 7: Runtime environment and resource limits used in the evaluation.

Component	Configuration
Host OS	Ubuntu 22.04.4 LTS, x86_64
CPU and memory	2× AMD EPYC 7H12; 128 physical cores / 256 hardware threads; 512 GiB RAM
Python	Python 3.13.2; shared virtual environment mounted into each run workspace
Core packages	pandas 3.0.1, numpy 2.4.3, scipy 1.17.1, scikit-learn 1.8.0, statsmodels 0.14.6, backtrader 1.9.78.123, pyarrow 23.0.1
Agent CLIs	OpenCode 1.14.28; OpenAI Codex CLI 0.124.0; Claude Code 2.1.119
Sampling and reasoning controls	No task-level override of temperature, top-p, or sampling seed. OpenCode runs use the built-in Build agent with the benchmark-specified backend model and no custom agent, variant, or sampling override; OpenCode therefore applies its model-specific temperature defaults, 0 for most models and 0.55 for Qwen models. Codex uses `model_reasoning_effort=xhigh`; Claude Code uses no explicit temperature, top-p, or `-effort` override in the benchmark runner
Attempt budget (timeout)	1800 seconds per agent attempt
Agent turn/budget caps	Claude Code runs use `-max-turns 80` and a per-run budget cap of USD 20 for Opus and USD 10 for Sonnet; OpenCode and Codex runs use the common wall-clock attempt budget
Checker budget	1800 seconds per checker-suite invocation
Agent scheduler concurrency	At most 5 concurrent agent attempts per provider pool
Checker runner concurrency	At most 5 concurrent checker-suite invocations
Thread caps	`OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`, and `NUMEXPR_NUM_THREADS` set to 1

The scheduler advances each `(instance, system)` pipeline independently through its repeated attempts; there is no global pass barrier across systems, benchmark instances, or attempt numbers. The model API call's concurrency limits are scheduling controls rather than model-performance settings. A slow or timed-out run blocks only its own `(instance, system)` pipeline, while other pipelines continue subject to the same per-pool limits. We do not tune sampling parameters per task. Because the evaluated scaffolds expose different controls, undocumented provider-side sampling details and hidden reasoning budgets are treated as scaffold/provider defaults unless explicitly listed in Table 7. The fixed thread caps prevent BLAS and NumExpr oversubscription when multiple agent attempts or checker suites execute concurrently.

F Additional Property-Level Discrimination Results

This section supports the breakdown analysis in § 4. We characterize all properties, show that the checker suite distinguishes different evaluated LLMs and agent scaffold configurations both across benchmark tasks and within each individual task, and quantify the no-result rate that drives the pass@1-pass@5 gap for several non-frontier agents. The panel covers 542 *(reproduction task, property)* pairs across 30 tasks and 9 agents, with 5 attempts per (agent, reproduction task) combination.

The properties can differentiate LLMs and agents. We classify each *(reproduction task, property)* pair by the agent's results (i.e., pass or fail) on it. A pair is *all-pass* if no agent fails it on any of its five attempts, *all-fail* if every agent fails at least once, and *discriminating* otherwise. Table 8 gives the breakdown. 78.8% of pairs are discriminating, only 5.4% are failed by every agent, and the remaining 15.9% are saturated. 81.0% Type-A and 77.4% Type-B properties are discriminating, respectively. Thus, both types are useful.

Table 8: Panel-level outcome of each *(task, property)* pair, excluding the auxiliary end-to-end metric property `A10`. Counts (and row shares) for the 9-agent panel evaluated on 5 attempts per task.

	all-pass	discriminating	all-fail	total
Type-A	33 (15.7%)	170 (81.0%)	7 (3.3%)	210
Type-B	53 (16.0%)	257 (77.4%)	22 (6.6%)	332
All pairs	86 (15.9%)	427 (78.8%)	29 (5.4%)	542

Most reproduction tasks contain discriminating properties, and weaker agents fail across more properties. Table 9 shows that this pattern also holds within individual strategy reproduction tasks. Every task contains properties that distinguish agents: some properties are passed by all agents, while others are passed by some agents but failed by others. Thus, the benchmark does not rely on a small number of hard checkers to separate agents.

We further measure, for each agent and task, how many properties the agent fails in at least one of its five attempts. The median task contains 18 properties. Stronger agents fail only a few properties per task: the median numbers are 3.0 for OpenCode GPT-5.5; 3.5 for Claude Code Opus; 4.0 for Codex GPT-5.5 and OpenCode Claude Opus; 4.5 for OpenCode GLM 5.1; and 5.0 for OpenCode Claude Sonnet. Weaker agents fail many more properties: 9.0 for OpenCode DeepSeek V4-Pro, 11.0 for OpenCode Qwen 3.6 Plus, and 13.0 for OpenCode MiniMax M2.7. Moreover, the failed properties are not identical across agents, suggesting that weaker performance reflects broader and more diverse reproduction errors rather than a single fixed bug pattern.

Table 9: Per-task property decomposition. *Cat* groups tasks into Momentum (Mom), Mean Reversion (MR), Factor (Fac), Volatility (Vol), Event-Driven (Evt), and Systematic Allocation (Sys). n is the number of properties evaluated for the task; the three count columns sum to n. The auxiliary end-to-end metric property `A10` is excluded.

Task	Cat	Paper	n	all-pass	discriminating	all-fail
T01	Mom	Moskowitz et al. (2012)	16	0	16	0
T02	Mom	Daniel and Moskowitz (2016)	16	1	14	1
T03	Mom	Novy-Marx (2015)	18	0	18	0
T04	Mom	Asness et al. (2013)	18	2	13	3
T05	Mom	Lou and Polk (2022)	17	0	16	1
T06	MR	Profitability of Pairs Trading	18	1	17	0
T07	MR	Focardi et al. (2016)	18	5	13	0
T08	MR	Chen et al. (Equity Pairs)	17	1	16	0
T09	MR	Guijarro-Ordonez et al. (2022)	14	0	13	1
T10	MR	Medhat and Schmeling (2022)	17	1	11	5
T11	Fac	Cohen et al. (2026)	19	5	10	4
T12	Fac	Arnott et al. (2023)	15	4	11	0
T13	Fac	Gu et al. (Empirical Asset Pricing)	22	5	16	1
T14	Fac	Neuhierl et al. (2024)	19	1	17	1
T15	Fac	Cong et al. (Panel Trees)	28	1	23	4
T16	Vol	Volatility-Managed Portfolios	17	1	16	0
T17	Vol	Betting Against Beta	18	1	17	0
T18	Vol	Conditional Volatility Targeting	17	3	14	0
T19	Vol	Downside Risk + Vol-Managed	17	1	16	0
T20	Vol	Option Momentum	18	5	13	0
T21	Evt	Savor and Wilson (Tale of Two Days)	18	4	13	1
T22	Evt	Lucca and Moench (2015)	17	10	7	0
T23	Evt	Hong et al. (2016)	18	3	15	0
T24	Evt	Guo (2025)	16	2	14	0
T25	Evt	Chen et al. (Short Selling Eff.)	19	1	17	1
T26	Sys	Kozak et al. (2020)	16	8	7	1
T27	Sys	Rapach et al. (2010)	17	9	8	0
T28	Sys	Kelly et al. (Universal Shrinkage)	23	1	22	0
T29	Sys	Factor Timing	18	8	10	0
T30	Sys	GMV Double Shrinkage	21	2	14	5

Table 10: All 36 hard Type-B properties with average fail rates above 50%.

Task ID	PID	Property Requirement Summary
Temporal consistency failures (17)		
T04	B2	Momentum signal for ALL asset classes = cumulative past 12-month raw return EXCLUDING most recent month, i.e. Σr_{t-i} for $i = 2..12$.
T02	B9	Use expanding-window OLS over lagged interaction terms
T05	B01	Use a 12-month momentum lookback and skip the most recent month
T09	B14	Keep the out-of-sample period fixed at 2002–2016
T11	B15	Use an 8-quarter LSTM sequence length
T11	B16	Use a rolling 28-quarter window with a 20/8 chronological split
T11	B26	Use out-of-sample predictions only for the signal
T14	B16	Compute OOS R^2 against an expanding-mean benchmark
T14	B2	Start prediction only after an initial expanding half-sample
T14	B21	Train PLS on lagged signals to predict next-period returns
T18	B4	Enforce a 10-year minimum warmup before volatility targeting
T21	B5	Estimate beta with a 12-month daily window (about 252 days)
T25	B10	Use mid-month short interest to ensure information-set availability
T25	B16	Predict returns from one-month-lagged SSE, not contemporaneous SSE
T25	B5	Detrend SSE with an expanding rather than rolling window
T25	B6	Winsorize only in-sample with expanding 1/99 cutoffs
T25	B7	Refit OLS in an expanding window for each OOS step
Definition implementation failures (16)		
T04	B11	Use inverse ex-post sample-std aggregation across markets
T02	B12	Model asymmetric volatility with GJR-GARCH
T11	B7	Use a single-layer LSTM architecture
T13	B1	Keep the NN3 architecture at 32–16–8
T14	B13	Use PLS to maximize covariance, not correlation
T15	B12	Use the regularized covariance formula $\Sigma + \gamma I$
T15	B13	Use exactly 20 boosted trees
T15	B14	Use the joint boosting split criterion with prior factors
T15	B16	Apply the full compound universe filter
T15	B22	Evaluate splits globally across all leaves
T15	B4	Construct exactly 61 firm characteristics
T24	B7	Scale volatility to the paper-specified 5.34% target
T30	B15	Compute the required Stieltjes transform functionals
T30	B17	Implement the bona fide loss formula exactly
T30	B18	Compute sample covariance with divisor n, not $n - 1$
T30	B19	Compute concentration ratio c correctly
Trading-direction failures (3)		
T10	B10	Preserve the long and short labels in STREV signals
T14	B5	Implement high-minus-low as top quintile minus bottom quintile
T25	B9	Enforce the negative-sign restriction on the OLS coefficient

Classification of hard paper-specific property failures. We classify the hard Type-B properties in two steps. First, among all paper-specific Type-B properties, we keep those whose average fail rate is above 50% across agents. This gives 34 hard Type-B properties. Second, we manually inspect each property by reading its checker, its name, and the corresponding evidence in the paper, and assign it to the most relevant failure category.

We have three categories. *Temporal consistency failures* cover errors in time ordering, lag structure, or information availability. *Definition implementation failures* cover cases where the submitted code uses the wrong paper-specified formula, parameter, model component, or architecture. *Trading-direction failures* cover cases where the code reverses or distorts the mapping from signals to long or short positions.

Missing runnable submissions explain part of the pass@1–pass@5 gap. We define a missing runnable submission as an attempt that produces no strategy implementation that the checker can execute. This includes empty submissions, where the agent exits without writing the strategy file, and timeouts, where the attempt exceeds the $1{,}800$s wall-clock budget. Figure 7 summarizes these missing runnable submissions by agent.

Across the 150 attempts per agent, Claude Code with Claude Opus, Codex with GPT-5.5, OpenCode with Claude Opus 4.7 and Claude Sonnet 4.6 produce runnable submissions in every attempt. Under OpenCode, the number of missing runnable submissions varies across model backends: GPT-5.5, DeepSeek V4-Pro, Qwen 3.6 Plus, MiniMax M2.7, and GLM 5.1 have 1, 4, 7, 19, and 80 such attempts, respectively.

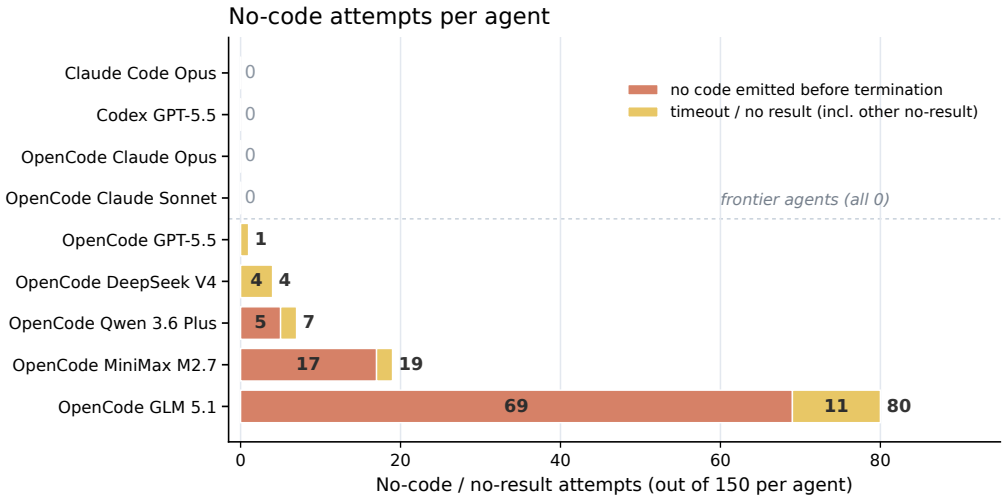


Figure 7: Per-agent missing runnable submissions across 1,350 trials. Bars split no-code terminations from timeout/no-result cases. Missing submissions are concentrated in OpenCode GLM 5.1 and MiniMax M2.7.

Most missing runnable submissions are no-code terminations rather than timeouts. Replaying the tool traces shows that, in these no-code attempts, the agents never invoke a file-writing tool to create the strategy file. Instead, they read files, run shell commands, update task lists, or terminate before issuing a write action. Figure 8 further decomposes the termination modes: among all missing/no-result attempts, 56 end in reasoning runaway, 8 remain in exploration or planning without a write, 25 terminate prematurely, 20 are timeout/no-result cases, and 2 are provider/API errors. The dominant failure mode differs across models. GLM 5.1 is dominated by reasoning runaway: many attempts exhaust the reasoning-token budget or continue repeated exploration without writing code. MiniMax M2.7 instead mostly exits prematurely after reaching the point where it appears ready to implement the strategy, but before writing the file.

These missing runnable submissions directly affect pass@1. For example, because GLM 5.1 produces no runnable code in roughly half of its single attempts, its pass@1 is limited by attempt-level reliability before implementation quality is even measured. pass@5 recovers part of this loss because at least one of the five attempts often produces runnable code.

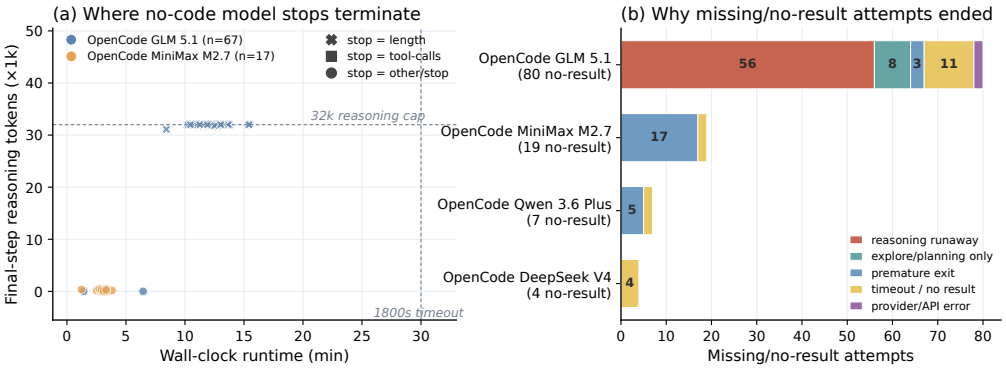


Figure 8: Trace-level reasons for missing/no-result attempts. Panel (a) shows where no-code model stops terminate; panel (b) decomposes missing/no-result attempts by termination reasons.

G Root Cause Annotation Process

Root-cause (RC) annotation is performed at the failed-checker-row level. Each row corresponds to one evaluated attempt, one failed property checker, the checker evidence, the final submitted

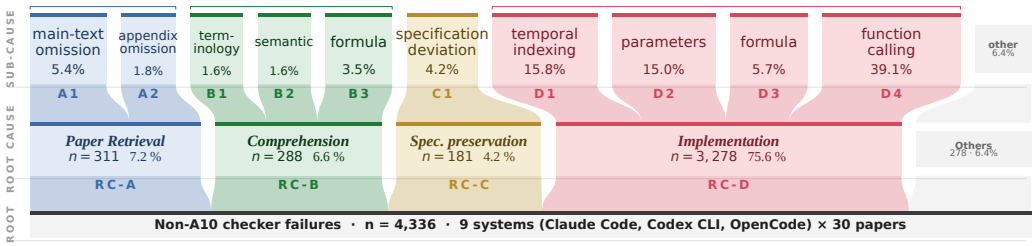


Figure 9: Root-cause distribution over all nine evaluated agents. Each non-A10 failed checker row is attributed to one of four primary RC stages when the trace supports a causal label; unresolved rows are reported as Others. RC-B and RC-D are further split into the fine-grained labels used for descriptive analysis.

strategy, and the archived agent trajectory. Annotators assign the earliest observable stage at which the trajectory departs from a faithful reproduction: Paper Retrieval (RC-A), Comprehension (RC-B), Spec. preservation (RC-C), and Implementation (RC-D). If the trace does not support attribution to one of these four stages, the row is marked `UNABLE_TO_DECIDE`; display figures group these abstentions as *Others*. We exclude A10 end-to-end consistency rows from the main attribution analysis because they may aggregate multiple upstream errors into a single metric failure; we use them only as a sensitivity stratum.

Human annotation. For the two strongest systems, *Claude Code with Claude Opus 4.7* and *Codex CLI with GPT-5.5*, all non-A10 failed checker rows are independently annotated by two CS PhD students and one CS postdoc (N=273 and N=367, respectively). Annotators assign labels from the archived agent trajectory, the relevant paper and Sample Instruction evidence, the checker evidence, and the submitted strategy code.

LLM as a judge and calibration. For the remaining seven OpenCode agents, we scale annotation with an LLM-as-judge pass over 3,696 non-A10 failed rows using the same evidence fields. We calibrate this extension with a 240-row blind audit sampled with seed 2026: 180 non-A10 calibration rows, 40 A10 sensitivity rows, and 20 boundary sentinels for cases where the trace does not support a confident A–D attribution. The three human annotators are blind to the machine label, confidence, and rationale. A human majority is obtained for $176/180$ rows; the LLM label has 77.8% exact agreement with the human majority, Cohen's κ=0.717, macro-F1 0.642, and weighted-F1 0.765.

Use of labels in the paper. The main text uses the fully human SOTA-2 labels for fine-grained RC analysis. The appendix reports the corresponding all-agent RC tree over all nine evaluated agents. LLM-assisted labels for OpenCode agents are used for aggregate primary-RC trends; A10 rows and non-SOTA sub-RC labels are treated as diagnostic rather than as the basis for strong claims. The *Others* bucket is retained for accounting completeness but is not interpreted as a causal stage.

Root-cause attribution model. The RC attribution model adapts the sequential error-analysis view from human-factors and software-error studies to the paper-to-code setting [76–78]. Recent agent-failure studies similarly motivate localizing failures from full execution traces and validating scalable LLM-as-judge annotations against human labels [79, 80]. Operationally, annotators run a top-down decision tree and stop at the *earliest observable breakdown* supported by the trace, paper evidence, and submitted code. The four primary stages are **RC-A Paper Retrieval**, where the required passage is not observed in the trace; **RC-B Comprehension**, where the passage is observed but its meaning is misread; **RC-C Spec. preservation**, where the paper requirement is understood but replaced by a prior, default, or simplification; and **RC-D Implementation**, where the stated intent is correct but the code does not preserve it. If the evidence does not support a causal label after this decision procedure, the row is assigned `UNABLE_TO_DECIDE`; this is a reserved abstention value, not a fifth root-cause class.

We further refine only the two stages where finer distinctions are most diagnostic. Within RC-B, we distinguish *terminology* errors (terms, units, or conventions), *semantic* errors (temporal or relational meaning), and *formula* errors (mathematical expressions or named methods). Within RC-D, we

separate *temporal indexing* because time ordering, lags, and lookbacks are especially consequential in quantitative finance; the remaining implementation labels distinguish *parameters* (wrong parameter values or choices), *formula* (incorrect algorithm-to-code mapping), and *function calling* (function names, argument types, composition order, and API corner cases). These labels are assigned only after the primary RC is fixed and do not override the earliest-breakdown rule.

Boundary rules. Adjacent RC labels are separated by observable agent behavior, not by the failed checker type. RC-A vs. RC-B is determined by access: if no trace quote, paraphrase, or tool-call range covers the required passage, the row is RC-A; once the passage is in context, an incorrect interpretation is RC-B. RC-B vs. RC-C is determined by fidelity after understanding: if the agent's stated interpretation contradicts the paper, the row is RC-B; if the interpretation is correct but the agent adopts a prior, default, simpler alternative, or unanchored specification choice, the row is RC-C. RC-C vs. RC-D is determined by whether the deviation appears before or during coding: a deliberate or silent substitution of the paper requirement is RC-C, whereas correct stated intent followed by inconsistent code, missing steps, indexing errors, or interface mismatches is RC-D. Ordinary untested code defects remain RC-D when the trace and submitted code identify a concrete implementation breakdown. When the A–D boundaries cannot be resolved from trace, paper, and code evidence, annotators abstain with `UNABLE_TO_DECIDE`.

H Task Card Index

This appendix summarizes the 30 reproduction tasks (T01–T30) that constitute QuantCode-Eval. Table 11 lists, for each task, the source paper (with citation), benchmark strategy category, evaluation period, the main bundled-data sources, the property breakdown (Type-A / Type-B / total), the golden-reference Sharpe and annualized return, and the line counts of the released golden reference and its checker suite. Aggregate corpus statistics are reported in Table 12. The full task datasets payload including paper list, property checker suite, golden reference, trade log, metadata, machine-readable card and experiment records—ships in the anonymous public code release at `https://anonymous.4open.science/r/QuantCodeEval-Anonymous`. A long-form walk-through of one representative task (T10, *Short-term Momentum*) is given in Appendix D.

Table 11: QuantCode-Eval task index. **Cat**: benchmark strategy category (Mom = momentum, MR = mean reversion, Fac = factor investing, Vol = volatility, Evt = event-driven, Sys = systematic allocation). **Sources**: bundled data providers / panels (CCM = CRSP/Compustat Merged; QF = *q*-factor library [15]; Welch–Goyal = equity-premium predictor panel [81]). **A/B/Tot**: implemented Type-A / Type-B / total properties. **Sh**: golden-reference Sharpe ratio. **AR**: annualized return (%). **GR**: golden-reference Python file lines of code. **Chk**: total lines of code summed across the released checker suite for the task.

ID	Ref	Cat	Sources	Period	A/B/Tot	Sh	AR%	GR	Chk
T01	[59]	Mom	futures/FX, yfinance	1999–2025	7/10/17	0.30	4.9	271	773
T02	[42]	Mom	WRDS, CRSP, French	1963–2013	6/11/17	1.16	22.3	733	2360
T03	[44]	Mom	WRDS, CRSP, Compustat	1975–2012	9/10/19	1.22	8.6	465	1361
T04	[58]	Mom	WRDS, CRSP, Compustat	1972–2011	9/10/19	1.71	17.9	885	1124
T05	[28]	Mom	WRDS, CRSP, French	1964–2010	8/10/18	0.43	10.1	653	3641
T06	[62]	MR	WRDS, CRSP, French	1963–2014	9/10/19	-0.24	-0.3	1190	2016
T07	[63]	MR	WRDS, CRSP	1989–2011	8/11/19	1.55	22.6	634	3474
T08	[29]	MR	CRSP, French	1968–2007	8/10/18	0.82	14.2	380	4212
T09	[60]	MR	CRSP, Derived, CCM	1998–2016	7/8/15	0.52	9.2	647	1392
T10	[61]	MR	WRDS, CRSP, CCM	1963–2018	8/10/18	1.02	15.2	425	1644
T11	[30]	Fac	WRDS, Compustat, Derived	1999–2024	10/10/20	0.06	0.5	1030	1133
T12	[41]	Fac	Compustat, French	1963–2015	8/8/16	0.87	3.6	285	959
T13	[23]	Fac	CRSP, Compustat, WRDS	1957–2016	10/13/23	0.52	4.3	874	2173
T14	[26]	Fac	CRSP, French, FRED	1963–2020	7/13/20	4.11	58.7	639	2640
T15	[43]	Fac	WRDS, CRSP, French	1981–2020	8/21/29	1.56	23.7	1136	3201
T16	[66]	Vol	French	1926–2015	6/12/18	0.51	9.9	177	900
T17	[67]	Vol	CRSP, French	1926–2012	9/10/19	0.74	9.9	462	1013
T18	[65]	Vol	French, yfinance	1972–2019	4/14/18	0.67	5.8	375	1645
T19	[64]	Vol	QF	1963–2018	7/11/18	1.76	6.9	300	2660

ID	Ref	Cat	Sources	Period	A/B/Tot	Sh	AR%	GR	Chk
T20	[31]	Vol	OptionMetrics, CRSP, French	1996–2019	8/11/19	1.07	85.8	405	2254
T21	[71]	Evt	CRSP, French, FRED	1964–2011	9/10/19	0.28	2.9	335	2064
T22	[70]	Evt	CRSP, Derived	1994–2011	7/11/18	0.84	2.8	208	824
T23	[69]	Evt	Compustat, CRSP, CCM	1988–2012	9/10/19	0.26	3.7	251	2176
T24	[19]	Evt	French, CRSP	1926–2021	7/10/17	0.50	9.2	257	2368
T25	[68]	Evt	CRSP, Compustat, WRDS	1974–2017	10/10/20	1.19	12.0	576	4619
T26	[72]	Sys	Kozak SCS, French	1973–2017	7/10/17	0.65	9.2	806	1584
T27	[74]	Sys	Welch-Goyal	1947–2005	9/9/18	0.34	10.6	503	1655
T28	[20]	Sys	JKP	1971–2022	7/17/24	1.92	19.2	470	2213
T29	[73]	Sys	WRDS, CRSP, Compustat	1974–2017	8/11/19	0.71	7.1	487	1356
T30	[13]	Sys	CRSP	2013–2021	9/13/22	0.47	8.5	586	2333

Table 12: Aggregate QuantCode-Eval corpus statistics (over 30 tasks). LoC counts are computed on the released task payloads at the versions linked from Table 11.

Quantity	Total	Mean	Median	Min	Max
Total properties per task	572	19.1	19	15	29
Type-A applicable per task	238	7.9	8	4	10
Type-B per task	334	11.1	10	8	21
Golden-reference LoC per task	16445	548	478	177	1190
Property Checker LoC (one checker)	61767	108	93	17	510
Property Checker LoC (sum per task)	61767	2059	2040	773	4619

I Limitations and Future Work

QuantCode-Eval focuses on reproducing academic quantitative trading strategies from research papers, which is a specific but practically important setting. Although our benchmark covers six strategy categories and 30 expert-selected papers, it cannot cover all papers in each category or other types of financial software, such as production trading infrastructure, discretionary investment research, real-time execution systems, or portfolio-risk management workflows. Therefore, our conclusions should be interpreted within the scope of paper-to-code quantitative strategy reproduction from 30 representative papers. Expanding the benchmark to more papers and financial task formats would be costly, but doing so is mostly a one-time cost and can further improve QuantCode-Eval.

Our empirical results should also be interpreted as a snapshot of the current LLM coding agents. Agent performance depends not only on the base model, but also on the surrounding coding-agent scaffold, tool-use policy, decoding configuration, and execution environment. As LLMs and coding-agent frameworks evolve, absolute model rankings and cost-performance tradeoffs may change. To support continued evaluation, we will release the benchmark artifacts, containerized environments, and reproduction instructions, and future versions of QuantCode-Eval can evaluate newly released LLMs, agent scaffolds, and additional finance papers.

J Licenses and Terms for Existing Assets Used in QuantCode-Eval

Source papers. Table 5 lists the 30 source papers used to define benchmark tasks, including title, authors, venue, and year. We do not redistribute these publisher-owned paper PDFs. Instead, we provide bibliographic metadata and Google Scholar links in the table.

Financial data. The financial data used in QuantCode-Eval come from different types of sources, and readers should obtain them according to the corresponding source type. First, some tasks require licensed financial databases, such as WRDS-hosted CRSP and Compustat. For these tasks, we do not redistribute raw data. Users should obtain the data through their own institutional subscriptions. Second, some tasks use publicly accessible research data libraries [82]. For these tasks, readers should download the data directly from the original public source and use the date range specified in the task. Third, some papers provide their own replication data or precomputed signals through

author websites or cloud-hosted files. For example, the Jensen-Kelly-Pedersen global factor data are distributed through the JKP data website and associated repositories [83]. Readers should obtain these data from the original JKP source. Across all cases, we do not claim ownership of third-party financial data and do not redistribute raw licensed datasets unless redistribution is explicitly permitted by the original provider.

Models and agent scaffolds. We evaluate both closed-source and open-weight LLMs under the coding-agent scaffolds described in § 4.1. Closed-source models are accessed only through their official APIs, and we do not redistribute their model weights. Similarly, proprietary coding-agent software is used only through the corresponding provider interfaces and is not redistributed. For open-source or open-weight components, we follow the corresponding licenses in our experiments. DeepSeek V4 model weights, including DeepSeek-V4-Pro and DeepSeek-V4-Flash, are released under the MIT License. GLM-5.1 model weights are also released under the MIT License. OpenCode, the open-source coding-agent scaffold used in our evaluation, is distributed under the MIT License. When these components are accessed through hosted inference services rather than local releases, we additionally follow the corresponding provider service terms.